Classification and Evaluation Metrics — Commentary

Lesson 4 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

Not examinable · reading time about 2 hours

Contents

Slide 1 — Lesson 4: Classification and Evaluation Metrics . . . 5

Part I — From a number to a decision **6**
Slide 2 — Before we start . . . 6
Slide 3 — Today: from a number to a decision . . . 6
Slide 4 — The question has changed . . . 6
Slide 5 — The fleet . . . 7
Slide 6 — 306 . . . 7
Slide 7 — What goes wrong if we just fit a line . . . 7
Question — why not simply clip the line's predictions to 0 and 1? . . . 8

Part II — The model **8**
Slide 8 — The sigmoid . . . 8
Question — slide 7's curve is flat over the first few sectors: do the first sectors say little? . . . 9
Question — so the logarithm was chosen to give the first sectors more weight? . . . 10
Slide 9 — Why the log-odds . . . 11
Question — why do odds help? They look more complicated than a probability . . . 11
Question — why set the log-odds equal to a line at all? . . . 11
Worked example — five probabilities, their odds and their log-odds . . . 12
Slide 10 — One reading, three different jumps . . . 12
Question — how does the figure show that evidence multiplies the odds? . . . 13
Question — is the starting value −6.20 the fleet's base rate? . . . 13
Question — does logistic regression assume the counters are independent? . . . 14
Slide 11 — Log-odds of p = P(y = 1 | x): a line . . . 14
Question — so the log-odds equal a linear model, and solving for p gives the probability of failure? . . . 14
Question — does logistic regression fit a line or a curve? . . . 14
Slide 12 — The model, and three properties of the sigmoid . . . 14
Slide 13 — The model, drawn: one neuron . . . 15
Question — is the perceptron the same thing as logistic regression? . . . 15
Slide 14 — What the coefficients mean . . . 15
Question — why does the slide begin with "standardised"? . . . 15

Question — is 1.80 per sector, or per unit of log(1 + count)? . . . 16
Question — six times the odds: is that six times as likely? . . . 16
Worked example — one coefficient, three drives . . . 17
Slide 15 — The truth, and what we recovered . . . 17
Question — why are the fitted coefficients not exactly the true ones, if the data is synthetic? . . . 17
Slide 16 — The intercept is the average drive's chance . . . 18
Question — is the intercept the log-odds when every x is zero? . . . 18
Question — how can the average drive have 0.2% when 3.8% of the fleet fails? . . . 18
Question — what does it take to reach a coin flip? . . . 18

Part III — Which cost to minimise **18**
Slide 17 — Fitting means minimising something, but what? . . . 18
Slide 18 — Step 1: each drive gets a p . . . 19
Question — B stayed healthy and has p = 0.1: is 0.1 the probability of what happened to B? . . . 19
Slide 19 — Step 2: how likely was the outcome? . . . 19
Question — why are the three probabilities multiplied? . . . 19
Slide 20 — Step 2 as formulas: one drive, then the likelihood . . . 19
Question — the top line is the likelihood of one example, the bottom the product over all? . . . 20
Slide 21 — Step 3: two settings, the likelier wins . . . 20
Question — why add a second setting and repeat the calculation? . . . 20
Slide 22 — Step 4: minus the log, a cost . . . 20
Question — so at setting 1 the likelihood is at its maximum and the cost at its minimum? . . . 21
Slide 23 — Surprise: minus the log of a probability . . . 21
Question — why is the vertical axis called "surprise"? . . . 21
Slide 24 — Cross-entropy, also called log loss . . . 22
Question — with y = 1, is the second term not $0 \times \log 0$, that is $0 \times (-\infty)$? . . . 22
Worked example — four drives, one confident mistake . . . 22
Slide 25 — Lesson 3's gradient, $\hat{y} = \sigma(w^\top x + b)$ . . . 22
Question — is ŷ here the prediction, the probability? . . . 23
Question — why does the gradient come out the same as linear regression's? . . . 23
Worked example — one step of gradient descent on three drives . . . 24
Slide 26 — So why not squared error? . . . 24
Question — log loss is larger, so gradient descent works better with it? . . . 25
Question — where does log loss's gradient with respect to z come from? . . . 25
Question — the figure says 1 − p, the notebook p − 1: is one of them wrong? . . . 25
Question — log loss's slope is $-1/p$, constant, and squared error's rises then falls? . . . 25
Slide 27 — And it is convex, which squared error is not . . . 26
Question — convexity is a third reason for log loss, stated without proof? . . . 26
Question — if the cost is convex, why is there no formula like the normal equation? 26
Question — which algorithm does `LogisticRegression` use? . . . 26

Part IV — The boundary, and notebook 1 **27**
Slide 28 — The decision boundary of a two-counter model . . . 27
Question — is it a two-feature model, or a slice of the six-feature one? . . . 27

Question — the drives above the line are flagged? . . . 27
Question — most failures are on the healthy side: is this a bad model? . . . 28
Slide 29 — Logistic regression is penalised too . . . 28
Question — why would a classifier need a penalty at all? . . . 28
Question — why does the penalty move the coefficient so little here? . . . 28
Question — is there a Lasso for classification too? . . . 28
Slide 30 — Which line? The one the cost chooses . . . 28
Question — so gradient descent finds a separating line, but not necessarily the best? 29
Question — how would the SVM's line differ? . . . 29
Slide 31 — Notebook 1, live . . . 29
Question — "Read the first two sigmoid lines together": which lines? . . . 29
Question — what do the three lines after the table say? . . . 29
Question — the formulas for odds and log-odds? . . . 29
Question — what does the gradient descent cell show? . . . 29
Slide 32 — Break . . . 30

Part V — Reading a classifier **30**
Slide 33 — Two models . . . 30
Question — this is leading to the confusion matrix? . . . 30
Slide 34 — Why everyone makes this mistake once . . . 30
Question — classes are almost always imbalanced, so accuracy is useless? . . . 30
Slide 35 — Four numbers instead of one . . . 31
Question — 33 missed: so there is room to improve the model? . . . 31
Slide 36 — The four outcomes, and what they cost . . . 31
Question — the costs depend on the application, not on the model? . . . 32
Question — and does a true positive cost nothing? . . . 32
Slide 37 — Precision and recall: look at the denominator . . . 32
Question — to maximise recall make FN zero, to maximise precision make FP zero, both by moving the threshold? . . . 32
Slide 38 — Our model, in those terms . . . 33
Question — specificity is the recall of the negative class, and one minus it is the false positive rate? . . . 33
Worked example — twenty drives, four of which fail . . . 33
Slide 39 — F1 = 2PR ÷ (P + R): why the harmonic mean . . . 34
Question — why the harmonic mean, and are there other reasons? . . . 34
Question — a miss costs 19 times a false alarm: which matches that? . . . 35
Slide 40 — Read the minority class, not the average . . . 35
Question — F1 for the negative class, and macro and weighted averages: new metrics? 35
Question — why is an arithmetic mean fine here, one slide after F1 rejected it? . . . 35
Slide 41 — Notebook 2, live . . . 35
Question — does the opening say the always-healthy model wins? . . . 35

Part VI — The threshold **36**
Slide 42 — The threshold is a choice nobody made . . . 36
Question — so 0.5 is only predict()'s default, and a precision of 77% means little without its threshold? . . . 36
Slide 43 — What the sweep actually costs . . . 37

Question — low threshold, almost every failure caught and many false alarms; at 0.9 no false alarms? . . . 37
Slide 44 — The precision-recall curve . . . 38
Question — how is the baseline computed? . . . 38

Part VII — Ranking: the ROC curve and the area under it **39**
Slide 45 — Comparing models without choosing a threshold . . . 39
Question — a new idea: judge the model by its ranking, not at one threshold? . . . 39
Slide 46 — The ROC curve . . . 40
Question — a steep start means a good model? . . . 40
Question — why is recall suddenly called the true positive rate? . . . 41
Slide 47 — What the area means . . . 41
Question — is the AUC a probability or a count? . . . 42
Worked example — the AUC of seven drives, by counting pairs . . . 43
Slide 48 — The scores behind the AUC . . . 43
Question — is the height the number of drives at that probability? . . . 44
Question — the failed drives to the left of healthy ones are what lowers the AUC? . 44
Slide 49 — What AUC does not tell you . . . 44
Question — divide by ten, same AUC: discrimination is not calibration? . . . 44
Question — an AUC of 0.95 means the probabilities are reliable? . . . 44
Slide 50 — Where ROC misleads . . . 45
Question — the ROC curves barely move while precision and AP fall? . . . 45
Question — why is the AUC 0.968 here and 0.949 on slide 47? . . . 45
Slide 51 — Why the two curves disagree . . . 45
Question — why does precision fall faster than recall when failures are removed? . . 46
Question — FPR unchanged, recall unchanged, so the ROC is fixed; and precision falls because the false positives grow? . . . 46
Slide 52 — The rule: report both . . . 46
Question — "its area": the area of the precision-recall curve? . . . 47
Worked example — average precision of a seven-drive ranking . . . 48

Part VIII — The threshold from money **48**
Slide 53 — Choosing the threshold from money . . . 48
Question — the threshold that minimises the real cost, and the costs come from the application? . . . 49
Question — must the threshold always come from costs? . . . 49
Slide 54 — Where the two costs cross . . . 49
Slide 55 — Where they cross, in one line . . . 50
Question — how is the formula reached? The probability is in the notes but not in the formula . . . 50
Question — on an imbalanced problem, does the threshold always belong below 0.5? 50
Worked example — invented costs: a false alarm €50, a missed failure €200 . . . 51
Slide 56 — The caveat you must state . . . 51
Question — we chose the threshold by looking at the test set: validation set, then test once? . . . 51
Question — if 0.078 is cheaper, why not deploy it? . . . 51
Slide 57 — What class weights do . . . 52
Question — "balanced" weights: the weights set from the costs of the two errors? . . 52

Question — so weights in the ratio k are always the same as a threshold of 1/(1 + k)? 52
Worked example — a weight ratio of 4 . . . 53
Slide 58 — Class weights: the same lever, pulled earlier . . . 53
Question — and resampling, or the synthetic minority oversampling technique? . . . 53
Slide 59 — Notebook 3, live . . . 54
Question — is the AUC bounded between 0.5 and 1? . . . 54
Question — why does the notebook print the scores, the pairs and the counts, and not just the AUC? . . . 54

Part IX — More than two classes (optional) 54
Slide 60 — Three classes: softmax (optional) . . . 54
Question — three probabilities adding to 1: where do they come from, and how is the class chosen? . . . 54
Worked example — softmax on three scores . . . 55
Slide 61 — More than two classes (optional) . . . 56
Question — how are precision and recall defined with three classes? . . . 56
Slide 62 — Macro against weighted, one last time (optional) . . . 57
Question — the weights are the number of drives per class, and the choice depends on the application? . . . 57
Slide 63 — What to take away . . . 57
Question — accuracy alone says little; precision for alarms, recall for failures; the threshold from costs; AUC for ranking? . . . 58
Slide 64 — Homework . . . 58

What this document is. A companion to the slides of lesson 4: for every slide, what it means, the steps it leaves implicit, and the questions it tends to raise, answered under the slide they belong to. It is a second explanation of the same material, in a different voice from the handout, which remains the reference for the derivations. It is **not examinable**: nothing here is needed for the exam that is not already in the handout and the notebooks.

Slide numbers and titles are the ones in the lesson's slide deck, in `Slides/`. Where a section mentions "the notes", it means the speaker notes stored in the slide deck, which PowerPoint shows below each slide. The questions are the ones that came up while the lesson was studied and reviewed; the three notebooks have their own questions, under the slide that introduces each of them.

Worked examples sit under ten slides (9, 14, 24, 25, 38, 47, 52, 55, 57, 60): a handful of invented values each, small enough to work through by hand. Their numbers are not the lesson's data; every one is computed by `commentary_examples.py`, beside this file, which also writes them here.

Slide 1 — Lesson 4: Classification and Evaluation Metrics

Lesson 3 predicted a number: the price of a house. Lesson 4 predicts a decision: will this disk drive fail in the next thirty days, yes or no? The lesson has two halves of very different weight.

- **The model**, about a third of the time: logistic regression, which is lesson 3's linear model with one change at the output, fitted with a different cost.

- **The evaluation**, the other two thirds: how to tell whether a classifier works, and how to report it without fooling anyone, yourself included.

The proportion is the message. Fitting a classifier is one line of scikit-learn; judging it honestly is where people go wrong, in coursework and in published work.

Part I — From a number to a decision

Slide 2 — Before we start

The discussion of Exercise 3 (Ridge, Lasso, and how to choose the penalty). The mistake to listen for is choosing the penalty by looking at the test error. It is a natural thing to do, because the test error is the number everyone was told to care about. But once the test set has helped choose anything, its score is no longer an honest estimate of performance on new data: the choice was fitted to it. Lesson 4 falls into the same trap on purpose, on slide 56, when a decision threshold is chosen by looking at the test set, and lesson 5 builds the machinery to avoid it.

Slide 3 — Today: from a number to a decision

Five items, and only the first is about the model. Two of the names will be new: the **receiver operating characteristic (ROC)** curve, which plots how fast a model catches failures against how fast it raises false alarms, and the **threshold**, the probability above which a drive is flagged. Why does evaluation come before the other classifiers of the course (k-nearest neighbours, support vector machines)? Because every one of them needs exactly this apparatus, and it is easier to learn it once, on the simplest model.

Slide 4 — The question has changed

$P(y = 1 \mid x)$ is read "the probability that y equals 1, given x". The bar means "given": to its right is what we know about the drive (its six counters, x), to its left what we want to know ($y = 1$, it fails). The two inequalities $0 \leq P \leq 1$ are the point of writing it this way: whatever the model outputs has to stay between 0 and 1.

The change from regression to classification forces two changes of machinery:

1. **The model**, because a straight line is unbounded and a probability is not.
2. **The evaluation**, because "how far off were we on average" means nothing when the answer is yes or no.

The interesting class is nearly always the rare one: fraud, disease, intrusion, equipment failure. That observation drives the whole second half of the lesson.

Slide 5 — The fleet

Self-Monitoring, Analysis and Reporting Technology (SMART) is the diagnostic data a drive keeps about itself. The six counters:

- **reallocated sectors**: patches of the disk the drive has retired and replaced with spare ones, the strongest warning;
- **spin retries**: times the motor needed a second attempt to spin the platters up;
- **read error rate** and **seek error rate**: the share of reads and of head movements that hit an error;
- **temperature** and **hours in service**.

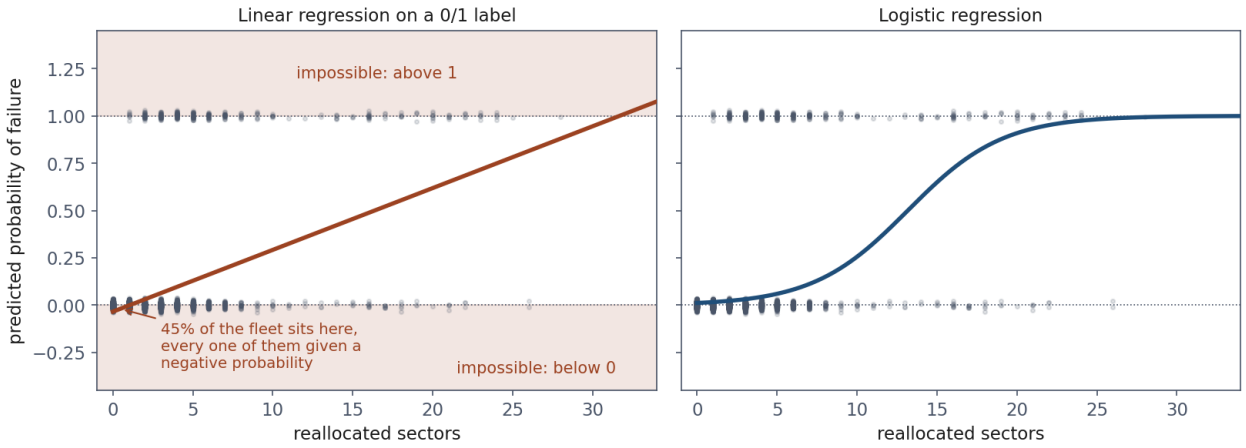
The data is synthetic, and that is deliberate: the coefficients that generated the labels are written down in `disk_data.py`, so every estimate can be checked against the truth. One column, `seek_error_rate`, was generated with a coefficient of exactly zero. It looks like telemetry and carries no signal: a decoy, revealed at the end of notebook 1.

`stratify=y` in `train_test_split` keeps the share of failures the same in the training and the test set. With failures this rare, an unlucky random split could otherwise put noticeably more of them on one side.

Slide 6 — 306

306 of 8,000 drives failed: $306 \div 8{,}000 = 3.8\%$, so 96.2% did not. The question the notes put to the room, "if a model must guess for each drive and wants to be right as often as possible, what should it say?", has an uncomfortable answer: "healthy", every time. That answer comes back after the break as an accuracy score, on slide 33.

Slide 7 — What goes wrong if we just fit a line



Left: least squares on the 0/1 label gives 45% of the fleet a negative probability. Right: the logistic curve never leaves the interval.

Ordinary least squares on a label that is 0 or 1 runs without complaint, and the result is not a probability: 3,606 of the 8,000 drives, 45% of the fleet, are given a negative one, and a drive slightly

worse than the worst in the data (32 sectors against 28) would get more than 1. No choice of slope and intercept fixes this, because a line is unbounded and a probability is not. The shape of the function has to change, not its parameters.

Question — why not simply clip the line's predictions to 0 and 1?

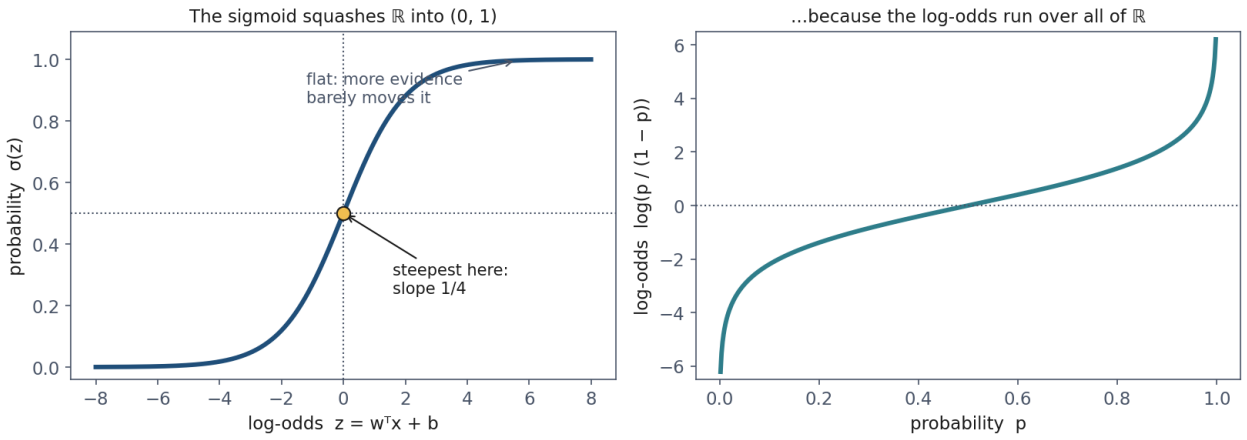
Clipping repairs the symptom, not the model. Three things stay wrong.

- **Certainty where there is none.** The 45% of drives below zero would all get exactly 0, a claim that they cannot fail. Some of them do fail; a model that called their failure impossible would be infinitely surprised by it (slide 23 makes that precise: minus the log of 0 is infinite).
- **No information in the clipped region.** Every drive below the cut gets the same answer, so the model can no longer say that one healthy-looking drive is riskier than another, which is what ranking (slide 45) needs.
- **Nothing to learn from.** Gradient descent follows the gradient of the cost, and by the chain rule each drive contributes (how the cost changes with p) times (how p changes with w and b). Where the line is clipped, p stays exactly 0 under a small change of w and b, so the second factor is 0 and the drive's contribution vanishes, whatever the cost. With log loss it is worse still: a failed drive given $p = 0$ costs $-\log 0$, which is infinite. Slide 26 shows a milder version of the same mechanism: squared error through the sigmoid loses its push as $p(1-p)$ goes to 0; clipping makes it exactly 0.

The sigmoid keeps every advantage of the line in the middle and bends smoothly at the ends, so every drive keeps an answer strictly between 0 and 1 that can be ranked and learned from.

Part II — The model

Slide 8 — The sigmoid



Left: the sigmoid against z, steep in the middle, flat at both ends. Right: the same relationship inverted, probability against log-odds.

The horizontal axis of the left panel is $z = w^\top x + b$, the number the linear model produces for a drive: think of it as the total evidence that the drive will fail. Slide 11 shows that z is exactly the log-odds.

The flat ends are the right behaviour, not a compromise, and it is the **total** evidence that has diminishing returns, not any one counter. Two more units of z from $z = 0$ take the probability from 0.50 to 0.88, a rise of 0.38; the same two units from $z = 6$ take it from 0.9975 to 0.9997, a rise of 0.0021, 178 times less. Once the model is nearly sure, more evidence barely changes its answer.

Question — slide 7's curve is flat over the first few sectors: do the first sectors say little?

> *The curve on the right of slide 7 barely moves over the first reallocated sectors and is steepest at 13.2. Is the sigmoid there the same one as here, centred at 0? And does the flatness mean the first sectors carry little information?*

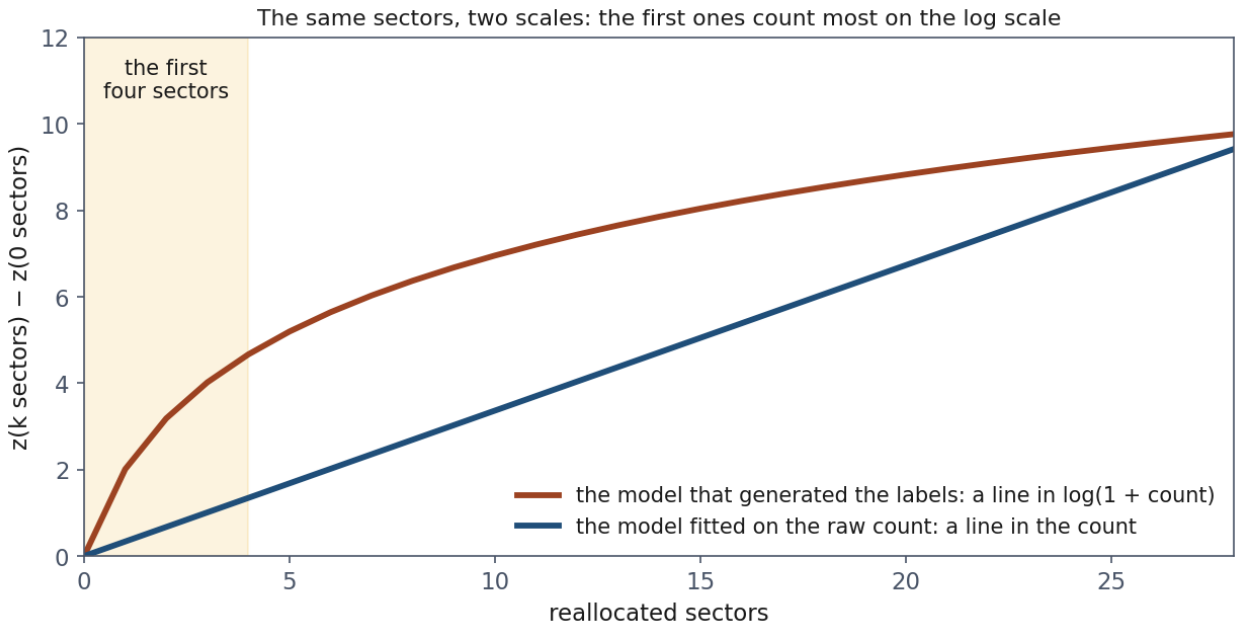
Two separate answers.

It is the same sigmoid, on a different axis. Slide 7's model has one input, the raw count of reallocated sectors as stored in the data, so its z is a straight line in the count: $z = 0.336 \times \text{count} - 4.43$. A drive with no sectors already has $z = -4.43$, deep in the flat left tail of this slide's curve; z reaches 0, the steep middle, where $0.336 \times \text{count} = 4.43$, at 13.2 sectors. Three drives make it concrete; each sits at the same height on both curves:

sectors	$z = 0.336 \times \text{count} - 4.43$	$p = \sigma(z)$
0	-4.43	0.012
13.2	0	0.500
28	4.99	0.993

Slide 7's curve is the stretch of this slide's curve between $z = -4.43$ and $z = 4.99$, drawn with sectors on the axis instead of z. It is one model seen twice, not two models: notebook 1 draws it as one curve with both axes, z along the bottom and sectors along the top.

The flat start is a fault of the model, not a fact about the drives. A drive with a few reallocated sectors is not safe. That model cannot see it, because it puts the raw count into a straight line, where every sector adds the same small step, 0.336. The labels were generated by a different rule: each counter goes through $\log(1 + \text{count})$ (the $+1$ keeps a drive with no sectors at $\log 1 = 0$ instead of $\log 0 = -\infty$), is standardised, and is multiplied by its true coefficient. On the log scale the first sectors weigh most: one more sector adds 0.693 to $\log(1 + \text{count})$ from 0 to 1, 0.087 from 10 to 11 and 0.035 from 27 to 28.



What the sectors add to z, for two drives identical except in the sector counter: $z(k) - z(0)$. The rust curve is the rule that generated the labels, the blue line the raw-count model. Over the first four sectors the rust curve climbs steeply.

The vertical axis needs one sentence. Take two drives identical in everything except the sector counter, one with k sectors and one with none, and subtract their z. Everything they share (the intercept, the other five counters, the mean subtracted by standardisation) cancels, so both curves start at 0 and can be compared. Four sectors add 4.67 to z under the generating rule and 1.35 under the raw-count line. As odds multipliers (slide 14 explains the conversion), four sectors multiply the odds of failing by 106 under the rule and by 3.8 under the line, about 28 times less. A model given a feature on the wrong scale under-rates exactly the early, common warnings, which is why the six-counter model from slide 12 onwards takes $\log(1 + \text{count})$ as its input.

Question — so the logarithm was chosen to give the first sectors more weight?

> *So log(count + 1) was used precisely to give more weight to the reallocated sectors from the very first ones, because they are the symptom of a problem?*

Yes, with two distinctions. The logarithm has **two roles**. In `disk_data.py` it is an assumption about how drives fail, built into the truth: the first sector a drive reallocates says something, the forty-first almost nothing new (handout section 1.1: going from 0 to 4 sectors is a 17 times larger move than going from 40 to 44). In the model it is feature engineering chosen to match that truth. We can check the choice here because the data are synthetic; with real data it comes from knowing the domain, or from seeing a counter that is skewed, with most values near zero and a long tail.

And the weight is **relative**. The logarithm does not boost the first sectors; it makes each further sector count less than the one before.

Slide 9 — Why the log-odds

Two steps, each removing one limit. The **odds**, $p \div (1 - p)$, remove the ceiling: a probability of 0.2 is odds of 0.25, "1 to 4", and as p approaches 1 the odds run off to infinity. Odds are never negative, so a floor remains at 0. The **logarithm** sends 0 to minus infinity, and the floor goes too. Nothing is lost: each probability has exactly one log-odds, and back.

Question — why do odds help? They look more complicated than a probability

Because evidence acts on them in the simplest possible way: it **multiplies** them. Take a **reading**, what one counter shows on one drive, such as "a high count of reallocated sectors", and suppose it shows up on 60% of the drives that will fail and on 10% of the healthy ones. Those two shares are made up for the example; their ratio, $0.6 \div 0.1 = 6$, is chosen close to the real effect of one standard deviation of reallocated sectors in this fleet, $e^{1.80} = 6.05$ (slide 14). A reading six times more common among failing drives multiplies the odds by 6, whatever they were before: of 1,000 drives at 50%, 500 fail and 500 do not, odds 1; among those showing the reading, 300 fail and 50 do not, odds 6. Slide 10 does the count in full. A probability cannot be treated that way: adding a fixed amount breaks at the edges (0.95 + 0.10 is 1.05), and multiplying it breaks too. Odds times 6 are always valid odds. The logarithm then turns each multiplication into an addition, and a sum of one term per feature is exactly what a linear model is.

Question — why set the log-odds equal to a line at all?

> *Slides 8 to 10 show the sigmoid and its inverse, but why should the log-odds equal a linear model? The range argument does not convince me: many functions map (0, 1) onto the whole line.*

Right: the range is a consequence, not the reason. The probit, built on the normal distribution, also maps $(0, 1)$ onto the real line and draws almost the same curve. The log-odds is chosen for three reasons, and the shape is not one of them:

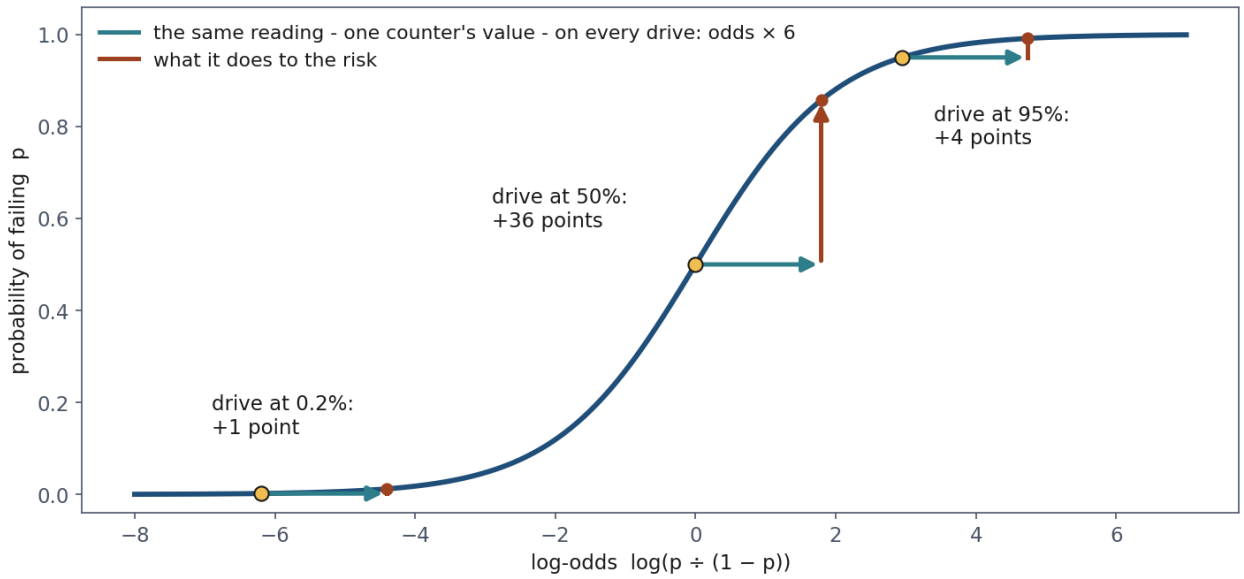
1. **Bayes' rule adds evidence on this scale.** The log-odds after seeing a drive's counters equal the log-odds before plus one term per counter (if the counters are independent given the class). One term per feature is a linear model.
2. **The coefficients read as odds multipliers** (slide 14). The coefficients are the weights w of the model, which arrives on slide 12; that is why the slide's bonus bullet speaks only of what each feature "will" add.
3. **The cost is convex and the gradient is simple** (slides 25 and 27).

Worked example — five probabilities, their odds and their log-odds

probability p	odds $= p \div (1-p)$	log-odds = log(odds), natural log
0.05	$0.05 \div 0.95 = 0.053$	log 0.053 = -2.94
0.2	$0.20 \div 0.80 = 0.250$	log 0.250 = -1.39
0.5	$0.50 \div 0.50 = 1.000$	log 1.000 = 0.00
0.8	$0.80 \div 0.20 = 4.000$	log 4.000 = 1.39
0.95	$0.95 \div 0.05 = 19.000$	log 19.000 = 2.94

Read the columns from left to right. The probability is squeezed into $[0, 1]$. The odds have no ceiling (0.95 already gives 19) but still stop at 0. The log-odds have neither limit, and they are symmetric: 0.2 and 0.8 sit at -1.39 and 1.39, the same distance either side of 0, which is where $p = 0.5$ lands.

Slide 10 — One reading, three different jumps



The same reading on three drives. The teal arrows, the odds multiplied by 6, have the same length on the log-odds axis; the rust arrows, what that does to the probability, are long in the middle and short at both ends.

The slide makes one point with a picture: the same piece of evidence moves three drives by the same step on the log-odds axis, and by very different amounts in probability. The evidence is a **reading**: what one counter shows on one drive, a particular value or range of values of one feature, such as "a high count of reallocated sectors". For the counting below it is a yes-or-no fact: the drive shows it or it does not. The reading here is six times more common among drives that fail than among healthy ones, so it multiplies the odds by 6. On the log-odds axis, multiplying the odds by 6 is adding $\log 6 = 1.79$, whatever the start: that is why the three teal arrows are equal. In probability the same factor gives three different jumps, the rust arrows:

drive	risk before, p	odds before = $p \div (1-p)$	odds after = odds $\times$ 6	risk after = odds $\div$ (1 + odds)	jump
average	0.2%	0.002	$0.002 \times 6 = 0.012$	$0.012 \div (1 + 0.012) = 1.2\%$	+1 point
uncertain	50%	$0.5 \div 0.5 = 1$	$1 \times 6 = 6$	$6 \div (1 + 6) = 86\%$	+36 points
nearly sure	95%	$0.95 \div 0.05 = 19$	$19 \times 6 = 114$	$114 \div (1 + 114) = 99.1\%$	+4 points

That contrast is why slide 11 puts the line on the log-odds and not on the probability.

Question — how does the figure show that evidence multiplies the odds?

From the figure, why should I understand that evidence multiplies the odds?

It does not show it; it assumes it and shows the consequence. The reason is a count. Imagine 1,000 drives at 50% risk, all like the drive in question, only so that percentages become counts: 500 will fail, 500 will not. Suppose the reading shows up on 60% of the failing drives and on 10% of the healthy ones; the two shares are an illustration, not measured on the fleet, and only their ratio, 6, matters. Among the drives showing it, 300 will fail and 50 will not. Our drive shows the reading, so it is one of those 350, and its odds go from 500 / 500 = 1 to 300 / 50 = 6.

The reading is a sieve that keeps a share of each group, 0.6 of the failing drives and 0.1 of the healthy ones, and shares multiply ratios: (0.6 $\times$ failing) $\div$ (0.1 $\times$ healthy) is 6 $\times$ (failing $\div$ healthy), whatever the start. Start from 950 and 50 and you get 570 and 5, odds from 19 to 114. The factor 6 is P(reading | fail) $\div$ P(reading | healthy), the **likelihood ratio**, and it belongs to the reading, not to the drive. That is Bayes' rule written with odds: odds after = odds before $\times$ likelihood ratio.

The probability cannot be treated the same way. It is failing $\div$ (failing + healthy): both groups sit in its denominator, shrunk by different shares, so how far it moves depends on where it started: $300 \div 350 = 86\%$ from 50%, $570 \div 575 = 99.1\%$ from 95%.

The figure then draws the consequence. The same factor of 6 is one fixed step on the log-odds axis, and on the probability axis it is 36 points at 50%, 4 points at 95% and 1 point for the average drive at 0.2%. Nobody chose the S-shape: it is what "evidence multiplies the odds" looks like drawn as a probability.

Question — is the starting value −6.20 the fleet's base rate?

No, and the difference matters. Bayes' rule starts from the odds **before any reading**, which come from the base rate alone: for this fleet $\log(306/7694) = -3.22$, a probability of 3.8%. The intercept $b = -6.20$ is something else: the log-odds of a drive whose counters all sit at the fleet average ($x = 0$ after standardising), a probability of 0.2%. Each counter's term in Bayes' rule depends on its reading; once the features are centred, the constant parts of all those terms fold, together with the base rate, into one number, b.

Why the average drive is so much safer than a drive drawn at random: the failures come from the minority of drives with bad telemetry, not from the average one. Slide 16 returns to this.

Question — does logistic regression assume the counters are independent?

No. Independence given the class was used only to **motivate** why evidence adds on the log-odds scale. Fitting a logistic regression does not require it: when two counters are correlated, the fitted coefficients share the weight between them. That is one of its advantages over naive Bayes, which does assume independence and double counts correlated evidence.

Slide 11 — Log-odds of p = P(y = 1 | x): a line

The left-hand equation is the modelling decision: put lesson 3's linear model on the log-odds. The right-hand side is algebra. Write $z = w^\top x + b$; exponentiate: $p/(1-p) = e^z$; collect p: $p(1+e^z) = e^z$, so $p = e^z/(1+e^z)$; divide top and bottom by e^z: $p = 1/(1+e^{-z})$. A check: a log-odds of 2.20 gives $1/(1+0.111) = 0.90$, and back, $0.9/0.1 = 9$ and $\log 9 = 2.20$.

Question — so the log-odds equal a linear model, and solving for p gives the probability of failure?

> *The log-odds are set equal to the evidence, a linear model, and solving for p gives the probability that a drive fails. Correct?*

Yes, with two precisions. p is not a vague "probability that the drive has problems": it is $P(y = 1 \mid x)$, the probability that this drive fails within thirty days, given its six counters. And the line has the form of lesson 3's but is **not fitted by least squares**: w and b are chosen by maximum likelihood, which slides 17 to 22 build.

Question — does logistic regression fit a line or a curve?

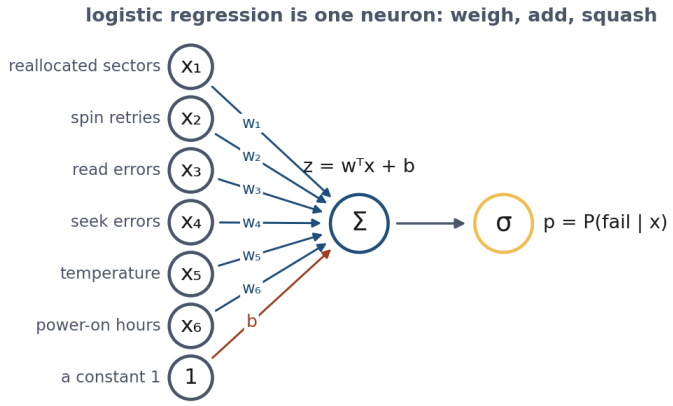
Both, on two different scales. It is a line in the log-odds, $z = w^\top x + b$, and a curve in the probability, $p = \sigma(z)$. The decision boundary (slide 28) is a line, because it is where $z = 0$.

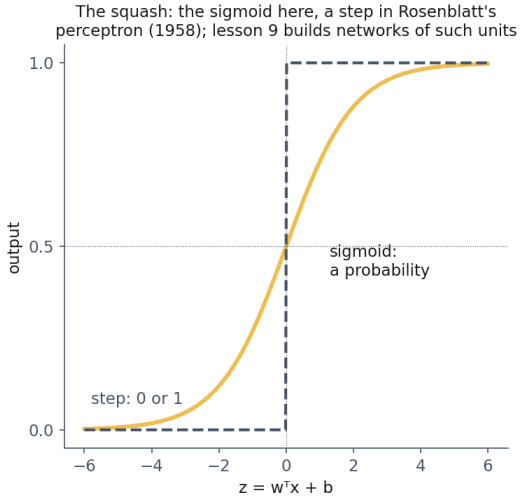
Slide 12 — The model, and three properties of the sigmoid

The first line is the whole model: take lesson 3's linear score $z = w^\top x + b$, read it as the log-odds, and turn it into a probability with the sigmoid. The **coefficients** w, also called **weights** (scikit-learn's `coef_`), and the **intercept** b (`intercept_`) are what fitting chooses. The other two lines are three properties of the sigmoid, each needed later; handout section 2.2 derives them.

- $\sigma(0) = \frac{1}{2}$: a probability of 0.5 is exactly $z = 0$, which is why the decision boundary of slide 28 is the line $w^\top x + b = 0$.
- $\sigma(-z) = 1 - \sigma(z)$: the probability of staying healthy is the same curve read from the other side, which gives the $y = 0$ term of the log loss (slide 24).
- $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, at most $\frac{1}{4}$. This one matters most. The derivative is largest in the middle, where it is $0.5 \times 0.5 = 0.25$, and decays to zero at both ends. That one fact does two jobs: it is why extra evidence changes little once the model is sure (slide 8), and why squared error fails with the sigmoid (slide 26), because the same factor ends up inside squared error's gradient.

Slide 13 — The model, drawn: one neuron





Each counter times its weight, plus the intercept (the weight on a constant input of 1), then the sigmoid: logistic regression drawn as a single unit.

In 1958 Frank Rosenblatt published the perceptron: the same weigh-and-add, followed by a hard step instead of the sigmoid, so its output was a label, 0 or 1, never a probability. It learned by its own rule, changing the weights only after a mistake. A neural network (lesson 9) is many such units side by side and one after another.

Question — is the perceptron the same thing as logistic regression?

Same diagram, different squash and different training. The step's slope is 0 everywhere except at $z = 0$, where it has none, so it gives gradient descent nothing to follow; the smooth sigmoid does. That is why neural networks use smooth activations and are trained by gradient descent on a cost like the log loss.

Slide 14 — What the coefficients mean

The slide reads the coefficients, the payoff of putting the line on the log-odds. Its first bullet is vocabulary: the coefficients w are also called weights (`coef_`), and b is the intercept (`intercept_`). The next two make one argument. With standardised features, one standard deviation more of a feature adds its coefficient to the log-odds, whatever the other features say; for reallocated sectors that is the true coefficient, 1.80. Adding to a logarithm multiplies what is inside it, so the odds are multiplied by $e^{1.80} = 6.05$: six times the odds, the teal step of slide 10. The questions below open the two words that hide the most: "standardised", and what one standard deviation is in sectors.

Question — why does the slide begin with "standardised"?

Why does the slide open with "standardised", and where do 1.80 and 6.05 come from?

A coefficient says how much z grows when its feature rises by one unit, and the raw units differ: a sector, a degree, an hour in service. Raw coefficients cannot be compared. `StandardScaler` rescales every feature to mean 0 and standard deviation 1, so from then on one unit means one standard

deviation for every feature. 1.80 is the **true** coefficient of reallocated sectors, the one `disk_data.py` generated the labels with (slide 15). 6.05 is $e^{1.80}$, because adding to a logarithm multiplies what is inside it: $\log(\text{odds}) + 1.80 = \log(\text{odds} \times e^{1.80})$. Checked on an average drive: its log-odds are -6.20, odds $e^{-6.20} = 0.0020$; one standard deviation more of sectors gives log-odds -4.40, odds 0.0123; the ratio is 6.05.

Question — is 1.80 per sector, or per unit of log(1 + count)?

Per standard deviation of the standardised $\log(1 + \text{count})$, and it takes four steps to turn that into sectors. Handout section 2.4 has the same steps and tables.

1. **What the model sees.** Not the count x but $\log(1 + x)$, standardised, so reallocated sectors contribute $1.80 \times (\log(1 + x) - 0.96)/0.62$ to z, where 0.96 and 0.62 are the mean and the standard deviation of $\log(1 + x)$ over the fleet.
2. **One standard deviation more is 0.62 more on** $\log(1 + x)$, not one sector more.
3. **Adding 0.62 to a logarithm multiplies what is inside it** by $e^{0.62} = 1.86$:

sectors now, x	$1 + x$	$(1 + x) \times 1.86$	sectors after one SD more
1	2	3.7	2.7
4	5	9.3	8.3
10	11	20.5	19.5

   Roughly, the count doubles, and each such step multiplies the odds by the same 6.05.

4. **So one extra sector is worth a lot at the start and little later.** Going from x to $x + 1$ adds $1.80 \times \log((x + 2)/(x + 1))/0.62$ to z:

one extra sector	added to z	odds multiplied by
$0 \to 1$	2.01	7.5
$1 \to 2$	1.18	3.2
$4 \to 5$	0.53	1.7
$10 \to 11$	0.25	1.3

So "one extra sector" does **not** multiply the odds by 6; what does is a near doubling of the count. Temperature and hours are standardised on their own values, so for them one standard deviation is an ordinary additive step.

Question — six times the odds: is that six times as likely?

Only for rare events. For a probability of 0.90 the odds are 9; multiplied by 5 they are 45, a probability of $45/46 = 0.978$: the odds rose fivefold, the probability eight points. Odds ratios multiply odds.

Worked example — one coefficient, three drives

Take a coefficient $w = 0.7$. One unit more of its feature adds 0.7 to the log-odds, so it multiplies the odds by $e^{0.7} = 2.01$: roughly doubles them.

Each row takes three conversions: odds $= p \div (1 - p)$; multiply the odds by 2.01; and back to a probability with $p =$ odds $\div$ (1 + odds).

probability before, p	odds before, $p \div (1-p)$	odds after, $\times 2.01$	probability after, odds $\div$ (1 + odds)	probability after $\div$ before
0.01	0.010	0.020	0.020	1.99
0.5	1.000	2.014	0.668	1.34
0.9	9.000	18.124	0.948	1.05

For the drive at 0.5: odds $0.5 \div 0.5 = 1$, times $e^{0.7}$ is 2.014, and back, $2.014 \div (1 + 2.014) = 0.668$.

The odds double for all three drives. The probability nearly doubles only for the rare event; at 0.5 it rises by a third, and at 0.9 it moves five points. "Twice the odds" and "twice as likely" agree only when the event is rare, which is why the habit of confusing them survives on this fleet and fails elsewhere.

Slide 15 — The truth, and what we recovered

The table is the check only synthetic data allows. The "True" column is what `disk_data.py` used to generate the labels; "Fitted" is what `LogisticRegression` estimated from the 6,000 training drives; the last column is e^{fitted}, each coefficient read as an odds multiplier as on slide 14. Read it top to bottom: the order of the six counters is recovered, the sizes are close (1.66 against 1.80 for sectors, 1.17 against 1.15 for spin retries), and the decoy `seek_error_rate`, generated at exactly 0, comes out at 0.02, a multiplier of 1.02. The fitted column is scikit-learn's default fit, which adds a small penalty (slide 27): that, and not the optimiser, is why notebook 1's hand-written fit differs from it in the second decimal.

Question — why are the fitted coefficients not exactly the true ones, if the data is synthetic?

Because the labels are still random. Each drive's label is **drawn**: the rule gives it a probability p, and the drive fails with probability p, so two drives with identical counters can get different labels. And the evidence is thinner than 6,000 suggests: what informs the coefficients is the 230 failures in the training set (306 in the fleet minus the 76 kept for testing). Synthetic data gives a known target, not an exact recovery from a finite sample. The ordering and the magnitudes are right, and the decoy was caught: generated at 0, estimated at 0.02, an odds multiplier of 1.02.

Would you have noticed the decoy if nobody had told you? That is the question the homework asks, on a dataset whose decoy is not revealed, and lesson 5 gives the tools to answer it reliably.

Slide 16 — The intercept is the average drive's chance

The intercept is what is left of the model when every feature is 0, and with standardised features 0 means average: the fitted -6.09 is the log-odds of a drive sitting at the fleet's mean on every counter. Through the sigmoid, $1 \div (1 + e^{6.09}) = 1 \div 442 = 0.0023$, a 0.2% chance of failing in thirty days. The fleet as a whole fails at 3.8% (slide 6), and the slide's third bullet says why the two differ: the failures come from the few drives far above average, not from the typical one. The questions below take the meaning of "zero", the gap between 0.2% and 3.8%, and what it takes to reach a coin flip.

Question — is the intercept the log-odds when every x is zero?

> *The intercept is the log-odds when all the x are zero?*

Yes, with the right meaning of zero. The x are standardised, so $x = 0$ is not a drive with no sectors and no errors: it is a drive sitting exactly at the fleet average on every counter ("average" on the scale the model sees: for sectors, the mean of $\log(1 + \text{count})$ is 0.96, about 1.6 sectors, below the arithmetic mean count of 2.2, because the logarithm compresses the long tail). The fitted intercept, -6.09, gives that drive $\sigma(-6.09) = 0.0023$, a 0.2% chance of failing.

Question — how can the average drive have 0.2% when 3.8% of the fleet fails?

Because the mean of the sigmoid is not the sigmoid of the mean. The sigmoid curves upwards in its left tail, so the few drives with a large z raise the average probability far more than the many near -6.2 can lower it, since no probability goes below 0. Two drives show it: at $z = -10.2$ and $z = -2.2$, 4 either side of -6.2, the probabilities are 0.00004 and 0.0998, so their mean is 0.050; the sigmoid of their mean z is 0.0020, 25 times less. Over the whole fleet, the true probabilities average 0.039, while the sigmoid of the mean z is 0.0020; the 3.8% observed (306 of 8,000) is what the random draw of the labels produced from that 0.039.

Question — what does it take to reach a coin flip?

$p = 0.5$ means $z = 0$, so the counters must add at least 6.2 to the intercept. Reallocated sectors alone, with the largest coefficient, need $6.2/1.80 = 3.44$ standard deviations of $\log(1+\text{count})$, which is 21.3 sectors: only 23 of the 8,000 drives have more. Any other drive gets there only with several counters bad at once.

This is what imbalance looks like in log-odds: almost the whole fleet sits far to the left of $z = 0$, so a threshold of 0.5 flags almost nothing. That is not the model being timid; it is the model being right about a rare event. Everything that looks like a failure of the classifier in the second half of the lesson traces back to this number.

Part III — Which cost to minimise

Slide 17 — Fitting means minimising something, but what?

The question is put to the room on purpose, because the usual first answers are wrong. Squared error applied to the sigmoid's output **is** differentiable, and it does produce a working model on easy

problems. The reason to reject it is subtler and comes on slide 26, after slides 18 to 22 have built, on three drives, the cost that logistic regression actually minimises.

Slide 18 — Step 1: each drive gets a p

Fitting means choosing w and b; call one particular choice a **setting**. Given a setting, the model turns each drive's counters into $p = \sigma(w^\top x + b)$. The three drives and their numbers are made up for the example. Slides 18 to 22 build the cost logistic regression minimises on these three drives, one step per slide: each drive gets a p (this slide), the probability of what happened (slides 19 and 20), two settings compared (slide 21), and minus the log, which turns the comparison into a cost (slide 22).

Question — B stayed healthy and has p = 0.1: is 0.1 the probability of what happened to B?

No. p is the probability of **failing**, for every drive, including the two that stayed healthy. The probability of what happened to B, staying healthy, is $1 - 0.1 = 0.9$, and that conversion is exactly the next slide.

Slide 19 — Step 2: how likely was the outcome?

Step 2 turns each drive's p into the probability the setting gave to **what actually happened**: p for a drive that failed, $1 - p$ for one that stayed healthy. A failed with $p = 0.8$, so 0.8; B stayed healthy with $p = 0.1$, so $1 - 0.1 = 0.9$; C stayed healthy with $p = 0.2$, so 0.8. The last row asks how probable it was that all three outcomes happened together, and multiplies: $0.8 \times 0.9 \times 0.8 = 0.576$. That product is the **likelihood** of setting 1.

Question — why are the three probabilities multiplied?

> *Why are the three probabilities multiplied? It should be introduced better.*

The last column is the probability of **what happened**: A failing (0.8), B staying healthy (0.9), C staying healthy (0.8). The probability that all three outcomes happen together is their product **if the drives are independent**, as for two coin tosses, $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$: one drive's outcome tells us nothing about another's, once their counters are known. Here $0.8 \times 0.9 \times 0.8 = 0.576$: under setting 1, what we observed had a 57.6% chance. That product is the **likelihood** of the setting.

Slide 20 — Step 2 as formulas: one drive, then the likelihood

The top line, $p^y(1-p)^{1-y}$, is the rule "p if $y = 1$, $1 - p$ if $y = 0$" in one expression: with $y = 1$ it gives $p^1(1-p)^0 = p$; with $y = 0$ it gives $1 - p$. The exponents are an if-statement. The bottom line multiplies those factors over the m drives ($\prod$ is to multiplication what $\sum$ is to addition).

Question — the top line is the likelihood of one example, the bottom the product over all?

Nearly. The top line is one drive's **factor**, the probability of what happened to that drive. The **likelihood** is the product on the bottom line, read as a function of (w, b) with the data held fixed: change the setting and every p changes, and the likelihood with them. That is why it is written $\mathcal{L}(w, b)$.

Slide 21 — Step 3: two settings, the likelier wins

Step 3 repeats step 2 for a second setting of w and b, one that gives the failed drive a low p, 0.3, and the healthy ones high ones, 0.6 and 0.5. Its likelihood is $0.3 \times 0.4 \times 0.5 = 0.060$, against setting 1's 0.576. The setting under which what happened was more probable wins: that comparison, made over every possible setting rather than two, is **maximum likelihood**.

Question — why add a second setting and repeat the calculation?

Because 0.576 on its own means nothing: there is no scale on which a likelihood is "high". It is only useful as a comparison. Setting 2 gives the failed drive a low p and a survivor a high one, and its likelihood is 0.060: what actually happened is $0.576/0.060 = 9.6$ times more probable under setting 1. That comparison is maximum likelihood. Fitting makes the same comparison over every possible setting, searched by gradient descent rather than one by one.

Slide 22 — Step 4: minus the log, a cost

Step 4 turns "largest likelihood" into "smallest cost", in two moves with a reason each; the line above the table says both.

The logarithm, for three reasons. It does not change the winner: the log is increasing, so the larger likelihood keeps the larger log, $\log 0.576 = -0.55$ against $\log 0.060 = -2.81$. It turns the product into a sum, $\log(abc) = \log a + \log b + \log c$, so each drive contributes its own term, and a sum differentiates one term at a time, which is where slide 25's gradient comes from. And it makes the computation possible at all: with every coefficient at zero each of the 6,000 training drives gets $p = 0.5$, and 0.5^{6000} is about 10^{-1806}, far below the smallest positive number a 64-bit float can hold, about 5×10^{-324}. The computer would store exactly zero.

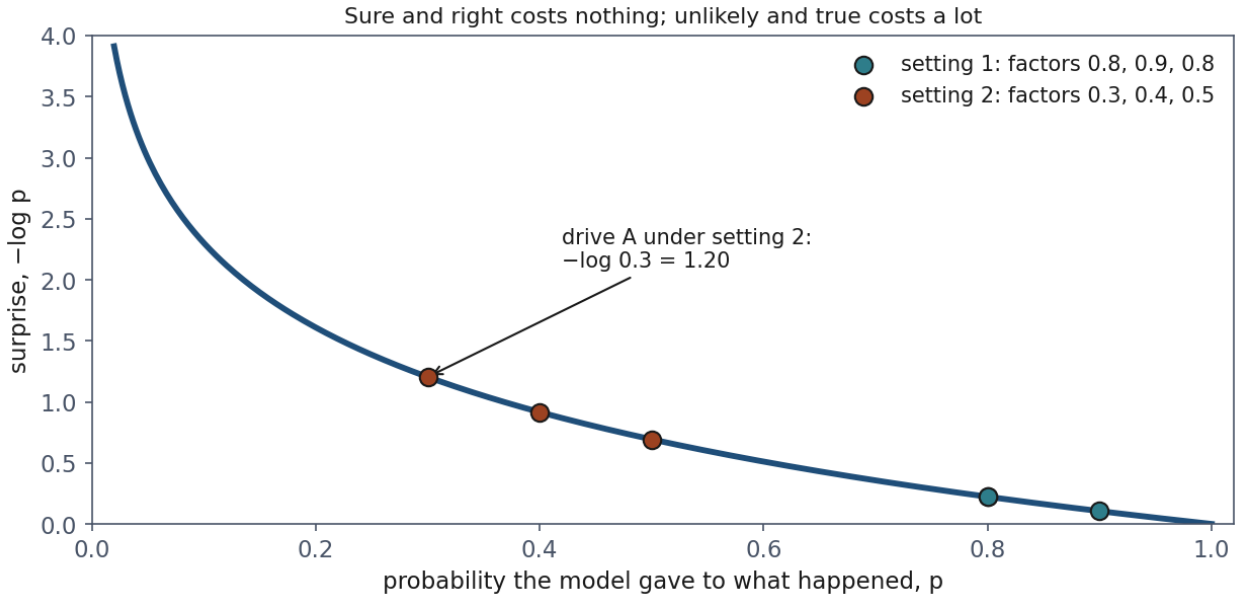
The minus. A probability is at most 1, so its logarithm is at most 0. The minus makes every term 0 or more and reverses the order, so the better setting now has the **smaller** number, 0.55 against 2.81, and fitting minimises, as gradient descent does. Because the log of a product is the sum of the logs, 0.55 is $-\log 0.576$ and 2.81 is $-\log 0.060$. Divided by the number of drives, the sum is the cost J, the **log loss**.

The logarithm is a convenience, not a new assumption: the best setting is the same with it or without it.

Question — so at setting 1 the likelihood is at its maximum and the cost at its minimum?

Half right. Where the likelihood is largest the cost is smallest, the same point, because $-\log$ reverses the order. But setting 1 is not that point: it only beats setting 2. Other choices of w and b would surely make these three outcomes more probable still. The maximum over every possible (w, b) is what fitting finds.

Slide 23 — Surprise: minus the log of a probability



Minus the log of the probability given to what happened: 0 for an outcome the model was sure of, unbounded for one it called nearly impossible. The six dots are the two settings' factors.

The slide draws one term of step 4: minus the log of the probability a setting gave to what happened, against that probability. The curve is 0 at the right, where the outcome was given probability 1, and climbs without limit at the left, where the outcome was called nearly impossible. The six dots are the six factors of slides 21 and 22: setting 1's three sit on the flat right-hand part, setting 2's higher, and drive A's 0.3 highest of all, at 1.20. That shape is why log loss punishes confident mistakes so hard.

Question — why is the vertical axis called "surprise"?

Because $-\log p$ behaves the way surprise does. It is zero when the model was certain and right ($-\log 1 = 0$), large when it had called the outcome unlikely ($-\log 0.001 = 6.9$), infinite when it had called it impossible. And for independent outcomes surprises add, because the log of a product is the sum of the logs, which is why three drives cost the sum of their three surprises. The name comes from information theory (Shannon's self-information) and is not examinable. The useful reading: the cost is the model's average surprise at what really happened, and fitting makes the data as unsurprising, which is to say as likely, as possible.

Slide 24 — Cross-entropy, also called log loss

Two lines, and the difference between them matters. The top line is the **loss** L on **one** drive: minus the log of the probability given to what happened, written as one expression because y is 0 or 1, so only one of the two terms is ever active. The bottom line is the **cost** J: the average of L over the m training drives, each with its own $p^{(i)} = \sigma(w^\top x^{(i)} + b)$. J is what fitting minimises, and the J that slide 25 differentiates. Check on drive B: $y = 0$, $p = 0.1$, so the first term vanishes and $L = -\log 0.9 = 0.11$, slide 22's number. Nobody designed this function: it is what maximum likelihood gives, slides 18 to 22 written in symbols.

Question — with y = 1, is the second term not 0 × log 0, that is 0 × (−∞)?

> *With $y = 1$ the second coefficient is 0, but the log of 0 is minus infinity: how is that justified?*

The coefficient is $1-y$, but the logarithm's argument is $1-p$, the model's probability, not the label. $p = \sigma(z)$ lies strictly between 0 and 1, so $\log(1-p)$ is finite and 0 times it is 0. Only p exactly 1 would give $\log 0$, and the sigmoid never reaches it. In a computer p can round to 1.0 when z is large, which is why code clips p away from 0 and 1: notebook 1 adds `1e-12` inside the log, and scikit-learn's `log_loss` clips too.

Worked example — four drives, one confident mistake

The loss of one drive is minus the log of the probability the model gave to what happened: p if the drive failed ($y = 1$), $1-p$ if it stayed healthy ($y = 0$).

drive	y	p, the P(fail)	probability of what happened: p if $y = 1$, $1-p$ if $y = 0$	loss $L =$ −log(probability of what happened)
A	1	0.9	0.9	−log 0.90 = 0.105
B	0	0.2	1 − 0.2 = 0.80	−log 0.80 = 0.223
C	0	0.1	1 − 0.1 = 0.90	−log 0.90 = 0.105
D	1	0.05	0.05	−log 0.05 = 2.996

The cost J is the average of the four losses: $(0.105 + 0.223 + 0.105 + 2.996) \div 4 = \mathbf{0.857}$. Drive D failed after the model gave it 0.05, and its loss alone, 2.996, is larger than the other three together. Log loss is dominated by confident mistakes, which is exactly where a fitting procedure should be pushed.

Slide 25 — Lesson 3's gradient, $\hat{y} = \sigma(w^\top x + b)$

In words: to see how the cost changes when w_j is nudged up, take each drive's error, the predicted probability $\hat{y}$ minus the label y, multiply it by the drive's value of feature j, and average. Gradient

descent steps w_j the other way. The symbols are exactly lesson 3's; the difference is hidden in $\hat{y}$, which is now the sigmoid of the line, and in the cost, log loss instead of squared error. The second line is the intercept's: the same formula with $x_j = 1$, because b is the weight on a constant input of 1 (slide 13), so $\partial J/\partial b$ is simply the average error. If the model over-predicts failure on average, b goes down.

Question — is ŷ here the prediction, the probability?

Yes: $\hat{y}$ ("y-hat") is $\sigma(w^\top x + b) = p$, the predicted probability of failing.

Question — why does the gradient come out the same as linear regression's?

Follow one drive through the chain rule, writing p for $\hat{y}$: the loss changes with p as $(p-y)/(p(1-p))$ (the denominator comes from the logarithms); p changes with z as $p(1-p)$, the sigmoid's derivative; z changes with w_j as x_j. Multiply: $p(1-p)$ cancels, and $(p-y)\,x_j$ is left. It is not a coincidence. Both models are **generalised linear models** fitted by maximum likelihood, each with its canonical link (the identity for Gaussian noise, the logit for a 0/1 outcome), and that pairing always gives the residual-times-feature form. The practical consequence: the same gradient descent code trains both, and only the line that computes $\hat{y}$ changes.

Worked example — one step of gradient descent on three drives

One standardised feature x, three drives, and the start of gradient descent: $w = 0$ and $b = 0$, so $z = 0 \times x + 0 = 0$ for every drive and $p = \sigma(0) = 0.5$.

drive	x	y	$p = \sigma(wx + b)$	error $= p - y$	error $\times$ x
1	2.0	1	0.5	$0.5 - 1 =$ -0.5	-0.5 $\times$ (2.0) = -1.00
2	-1.0	0	0.5	$0.5 - 0 = 0.5$	0.5 $\times$ (-1.0) = -0.50
3	0.5	0	0.5	$0.5 - 0 = 0.5$	0.5 $\times$ (0.5) = 0.25

The two gradients are the slide's two formulas, averages over the $m = 3$ drives:

- $\partial J/\partial w$ = average of the last column = (-1.00 − 0.50 + 0.25) ÷ 3 = **-0.4167**
- $\partial J/\partial b$ = average of the errors = (-0.5 + 0.5 + 0.5) ÷ 3 = **0.1667**

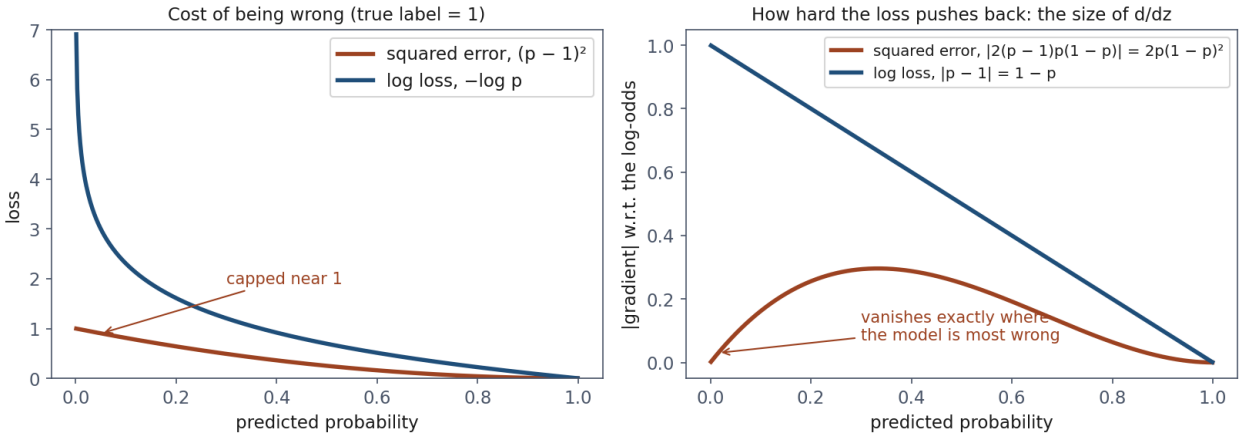
The update rule is lesson 3's: step against the gradient, new value = old value − α × gradient. With a learning rate $\alpha = 0.5$:

- $w = 0 - 0.5 \times (-0.4167) = 0.2083$
- $b = 0 - 0.5 \times 0.1667 = -0.0833$

What the signs mean: $\partial J/\partial w$ is negative, so the cost falls if w rises, and the step raises w; $\partial J/\partial b$ is positive, because on average the model over-predicts failure (two of the three drives are healthy and were given 0.5), so the step lowers b.

The failed drive, with a large positive x, pulls w up. So does the healthy drive at $x = -1$: a larger w lowers its p, which is what a healthy drive needs. Only the healthy drive at $x = 0.5$ pulls w down, and weakly, because its x is small. That is lesson 3's weighted vote, with p in place of the straight line.

Slide 26 — So why not squared error?



Both panels follow a drive that failed. Left, the costs: squared error is capped near 1, log loss is not. Right, the push gradient descent receives, with respect to z: squared error's vanishes exactly where the model is most wrong.

The slide answers the question slide 17 left open, on one failed drive followed as the model grows more wrong. The left panel compares the two costs: squared error, $(1-p)^2$, never exceeds 1; log loss, $-\log p$, grows without limit as p goes to 0. The right panel is the argument: the push gradient descent receives, the derivative with respect to z. Log loss's is $p-1$, largest exactly when the model is most wrong; squared error's carries the factor $p(1-p)$ of slide 12 and dies as p goes to 0. Under squared error a model that is confidently wrong stops learning where it most needs to.

Question — log loss is larger, so gradient descent works better with it?

> *Left panel: squared error tops out at 1, log loss goes to infinity. So the cost is larger and gradient descent probably works better?*

The reading of the left panel is right; the conclusion is not. Gradient descent uses the **gradient**, not the size of the cost: multiply a cost by 100 and its minimum is in the same place. The argument is the right panel. For a drive with $y = 1$, squared error's gradient with respect to z is $2(p-1)\,p(1-p)$, and the factor $p(1-p)$ goes to zero as p goes to zero: at $p = 0.001$ the push is 0.002, against log loss's 0.999. The wronger the model gets, the less squared error asks it to change.

Question — where does log loss's gradient with respect to z come from?

By the chain rule, in four steps (handout section 3.4 writes them out).

1. How the loss changes with p: $\partial L/\partial p = -[\,y/p - (1-y)/(1-p)\,]$, because the derivative of $\log p$ is $1/p$ and that of $\log(1-p)$ is $-1/(1-p)$.
2. The chain rule: multiply by the sigmoid's derivative, $dp/dz = p(1-p)$ (slide 12).
3. Multiply into the bracket: $p(1-p) \div p = 1-p$ and $p(1-p) \div (1-p) = p$, so $\partial L/\partial z = -[\,y(1-p) - (1-y)p\,] = -[\,y - yp - p + yp\,] = p - y$. The two yp cancel.
4. The two labels: for a failed drive, $y = 1$, the gradient is $p-1$; for a healthy one, $y = 0$, it is p.

For squared error the same chain gives $2(p-1) \times p(1-p)$, and nothing cancels the factor $p(1-p)$.

Question — the figure says 1 − p, the notebook p − 1: is one of them wrong?

Neither. The gradient is $p-1$, negative for a failed drive, which makes gradient descent raise z. The right panel draws its **size**, as its vertical axis says ("|gradient|"): $|p-1| = 1-p$, and for squared error $|2(p-1)p(1-p)| = 2p(1-p)^2$. The size is what the panel compares: how hard each cost pushes, not in which direction.

Question — log loss's slope is −1/p, constant, and squared error's rises then falls?

The second half is right: squared error's push peaks in the middle and is zero where its cost is largest. The first half mixes the two panels. $-1/p$ is the slope of the **left** blue curve, the derivative of $-\log p$ with respect to p. The right panel differentiates with respect to z, and there log loss's gradient is $p-1$, of size $1-p$: the straight line from 1 down to 0 in its legend. The chain rule joins the panels: $-1/p$ times the sigmoid's derivative $p(1-p)$ gives $-(1-p)$. The $1/p$ cancels the sigmoid's p, which is why log loss's push does not die; squared error has no $1/p$ to cancel it.

Slide 27 — And it is convex, which squared error is not

The third reason for log loss, and the one that makes fitting safe: as a function of the weights it is **convex**, bowl-shaped with no false bottoms, so any local minimum gradient descent finds is the global one. Squared error through a sigmoid is not convex, and the slide's third bullet gives the reason from slide 26's right panel: as a function of z its slope starts at 0, goes negative and climbs back to 0, while a convex curve's slope can only grow. What convexity does not give is a formula: logistic regression has no normal equation and is solved iteratively, and by default scikit-learn does not use our gradient descent to do it (the last question below).

Question — convexity is a third reason for log loss, stated without proof?

Yes; the handout (section 4.3) proves it. Append a constant 1 to every x so that b is one more weight; the matrix of second derivatives is $H = \frac{1}{m}\sum_i p_i(1-p_i)\,x_i x_i^\top$, and for any direction u, $u^\top H u = \frac{1}{m}\sum_i p_i(1-p_i)(x_i^\top u)^2 \geq 0$: a positive number times a square. No direction curves downwards, so any local minimum is global. For squared error the right panel of slide 26 is already the proof that it is not convex: as a function of z its slope is 0 far left, negative in the middle, 0 again far right, and a convex function's slope can only grow.

Convexity promises less than it sounds. It does not promise a **unique** minimum, nor even a finite one: if a line separates the two classes perfectly, the cost keeps falling as the weights grow, and it is the penalty scikit-learn adds by default that keeps them finite.

Question — if the cost is convex, why is there no formula like the normal equation?

Convexity is about the shape of the problem, not about solving it in closed form. Setting the gradient to zero gives $X^\top(\sigma(Xw+b)-y)=0$, which is not linear in w: the sigmoid is in the way. There is no normal equation for logistic regression; it is solved iteratively, which is why `LogisticRegression` has a `max_iter` argument and `LinearRegression` does not, and what a convergence warning is about.

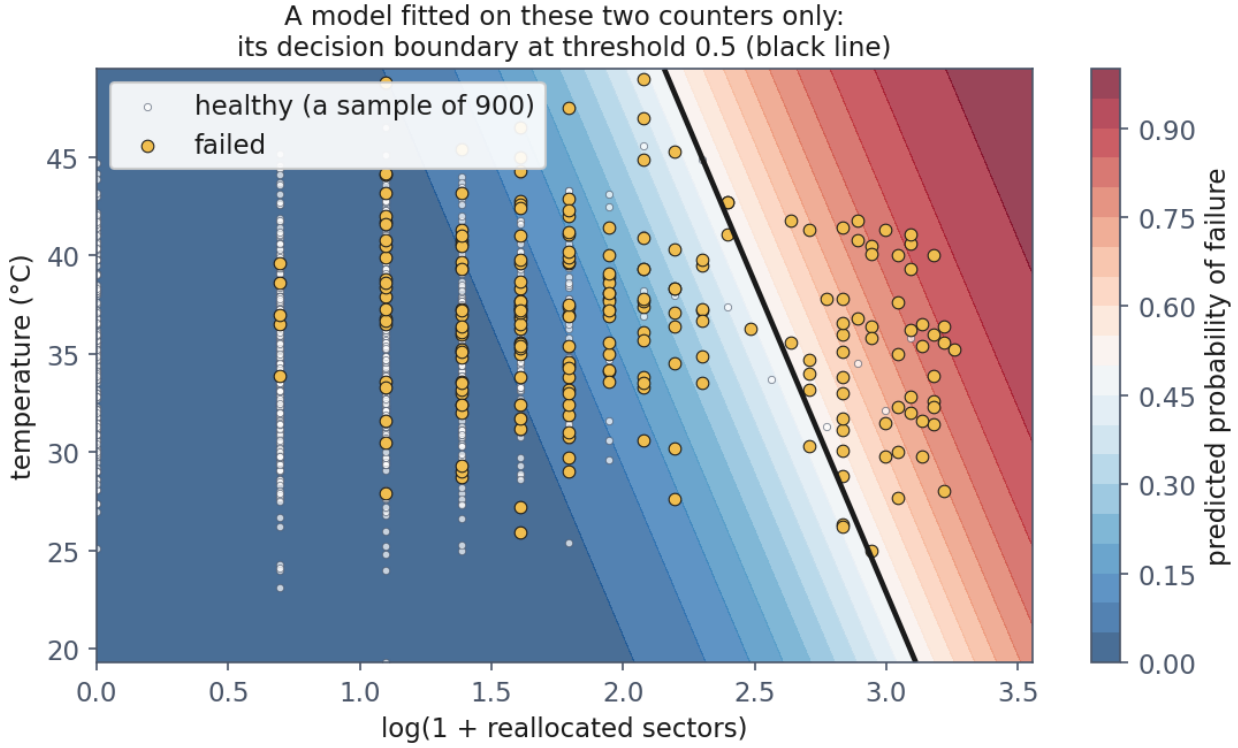
Question — which algorithm does `LogisticRegression` use?

Not the gradient descent of notebook 1. The default is `solver="lbfgs"`, limited-memory Broyden–Fletcher–Goldfarb–Shanno (L-BFGS), a quasi-Newton method: it estimates the curvature of the cost from the last few gradients and uses it to choose both the direction and the length of each step, so there is no learning rate to set. On notebook 1's data it stops after 15 iterations, where the hand-written descent ran 4,000. The other solvers (`newton-cg`, `newton-cholesky`, `sag`, `saga`, `liblinear`) reach the same minimum by other routes; `saga` is the one that supports an L1 penalty.

And it does not minimise the log loss alone: the default adds an L2 penalty on the coefficients, as lesson 3's ridge did, with `C=1.0`, where C is the inverse of the penalty strength, so a smaller C penalises harder. That penalty is why notebook 1's hand-written coefficients and scikit-learn's differ in the second decimal, 1.685 against 1.664 for reallocated sectors; refitted with `penalty=None`, scikit-learn gives 1.690 and the gap closes. The default is wanted, not a bug: when a line separates the classes perfectly, unpenalised weights grow without limit, and lesson 5 chooses C by cross-validation. Handout section 4.3.

Part IV — The boundary, and notebook 1

Slide 28 — The decision boundary of a two-counter model



A separate model, fitted on two counters only so that it can be drawn. The black line is where the predicted probability is 0.5; the model flags the red side.

The figure draws a model of two counters, reallocated sectors and temperature, as a map: each point is a training drive, the shading is the predicted probability, and the black line is where it equals 0.5; drives on the red side are flagged. The boundary is **straight**, because at threshold 0.5, $\sigma(z) = 0.5$ exactly when $z = 0$, and z is linear in the features. The curve is in the probabilities, not in the boundary.

Question — is it a two-feature model, or a slice of the six-feature one?

A separate model, fitted on reallocated sectors and temperature only. A slice of the six-feature model would need the other four counters fixed at some value. The white points are a random sample of 900 healthy training drives, and every training failure is drawn, so gold looks far more common than it is. The vertical stripes come from the sector counter being an integer: every drive with the same count sits at the same $\log(1 + k)$.

Question — the drives above the line are flagged?

A line has two sides, not an above and a below. The model flags the side where $w^\top x + b > 0$, here the red side, to the right. The line is almost vertical because sectors weigh far more than temperature, which only tilts it.

Question — most failures are on the healthy side: is this a bad model?

The plot cannot tell. It says nothing about whether the probabilities are calibrated. What it shows is that at 0.5 most of the drives that failed sit on the side the model calls healthy, and that moving the threshold would trade misses for false alarms without refitting anything. Telling a bad model from a badly chosen threshold is the rest of the lesson. (The six-counter model does better at the same 0.5 and still misses 33 of the 76 test failures.)

Slide 29 — Logistic regression is penalised too

The slide says that lesson 3's regularisation is not only for regression. `LogisticRegression()` adds an L2 penalty, the sum of the squared coefficients, to the log loss without being asked: lesson 3's Ridge, on a classifier. Its strength is written C, the inverse of lesson 3's λ, $C = 1/\lambda$, so a smaller C means a stronger penalty; the default is $C = 1$. On the lesson's 6,000 training drives it barely shows: the coefficient of reallocated sectors is 1.664 with the default and 1.690 with `penalty=None` (notebook 1, section 6).

Question — why would a classifier need a penalty at all?

Because without one the log loss may have no minimum. Take two classes a line separates perfectly: every drive is already on the right side, and doubling the weights still lowers the cost, because every probability moves closer to the 0 or 1 it should be. Doubling again lowers it again, for ever. Convexity (slide 27) promised that any minimum is global; it did not promise that one exists. The penalty charges for large weights and gives the cost a bottom.

Question — why does the penalty move the coefficient so little here?

The log loss is a sum over the 6,000 drives, the penalty a fixed charge. With that much data the drives outvote the penalty, and the coefficient moves in the second decimal, 1.690 to 1.664. On a handful of drives the same default can cut a coefficient by a third (Consolidation atom $00 \cdot 11$ shows it on eight).

Question — is there a Lasso for classification too?

Yes: `penalty="l1"`, with a solver that supports it (`solver="saga"` or `liblinear`), puts lesson 3's diamond on the log loss and sets some coefficients exactly to zero. In both cases C is a setting, not something the fit learns, so it is chosen by cross-validation, which lesson 5 builds.

Slide 30 — Which line? The one the cost chooses

The slide answers the question slide 28 leaves open: is that line the best one, or just one where gradient descent happened to stop? The line does not depend on the algorithm. Log loss plus the penalty is convex, so gradient descent, scikit-learn's solver or any other method end at the same coefficients. It depends on the cost: every drive pulls on the line in proportion to its error, $p - y$, the far and correctly classified ones too, weakly. When many lines separate the classes, "best" only means something once a criterion is named, and lesson 6's support vector machine (SVM) names a different one: the widest empty strip around the line.

Question — so gradient descent finds a separating line, but not necessarily the best?

It is a natural thought, and it is true of the perceptron, Frank Rosenblatt's rule of 1958, which stops at the first line that puts every training point on its side. Gradient descent on log loss does not stop there: it keeps going until the cost stops falling, and because the cost is convex, where it stops is the minimum. The line is the best by that cost. The question worth asking is which cost.

Question — how would the SVM's line differ?

The SVM keeps the line with the widest empty strip between the classes, and that line is fixed by the few points nearest the border alone: move any other point, without crossing into the strip, and the line does not change. Logistic regression listens to every drive, the SVM only to the ones on the edge. Two sensible criteria, in general two different lines; lesson 6 draws them.

Slide 31 — Notebook 1, live

Notebook 1 runs the first half of the lesson in code: the line on a 0/1 label, the sigmoid, the two losses and their gradients, gradient descent in fifteen lines of NumPy, the decision boundary, and the coefficients recovered against the truth. The questions below came up while working through it.

Question — "Read the first two sigmoid lines together": which lines?

The printout of the odds table ends with three lines under "sigmoid undoes the log-odds". The first two are the ones meant: `sigmoid(0) = 0.500`, a coin flip, and `sigmoid(-6.2) = 0.0020`, the average drive. The gap of 6.2 in z between them is what the evidence must cover before the model raises an alarm at 0.5, which is the whole story of an imbalanced problem (slide 16). The third line, `sigmoid(+2.2) = 0.900`, reads the table backwards: a log-odds of 2.2 is a probability of 0.90.

Question — what do the three lines after the table say?

They walk the chain of slide 16 with the fleet's own numbers.

- **The average drive**: the mean of $\log(1+\text{count})$ over the fleet is 0.96, about 1.6 sectors, below the arithmetic mean count, 2.2.
- **A coin flip on sectors alone**: $6.20/1.80 = 3.44$ standard deviations of $\log(1+\text{count})$, which is 21.3 sectors; 23 of the 8,000 drives have more.
- **Mean of the sigmoid against the sigmoid of the mean**: 0.039 against 0.0020, and the two-drive example of slide 16 shows why.

Question — the formulas for odds and log-odds?

For a probability p: odds $= p/(1-p)$ and log-odds $z = \log(p/(1-p))$, with the natural logarithm. Going back is one line: $e^z = p/(1-p)$, so $p = e^z/(1+e^z) = 1/(1+e^{-z}) = \sigma(z)$. That is what "the sigmoid is the inverse of the log-odds" means.

Question — what does the gradient descent cell show?

The log loss falling from 0.6931 at the start to 0.0692 after 4,000 iterations. The starting value is not arbitrary: with every weight at zero every drive gets $p = 0.5$, and the log loss of a coin flip is

$-\log 0.5 = 0.6931$, whatever the labels. The hand-written fit and scikit-learn then agree with the truth (slide 15), and with each other to about 0.02. The next cell shows where that gap comes from: scikit-learn's default penalty. Refitted with `penalty=None`, it gives 1.690 for reallocated sectors against the hand-written 1.685 (slide 27, last question).

Slide 32 — Break

Ten minutes. The first slide after it is the trap of the lesson, and it works best on a room that has not been thinking about it.

Part V — Reading a classifier

Slide 33 — Two models

The number to remember from lesson 4. A model that could be written `return 0`, with no data and no learning, scores **96.20%** on the 2,000 test drives, because 1,924 of them are healthy. The logistic regression, which finds 43 of the 76 failures, scores 97.70%. One and a half points separate a real model from a worthless one.

Question — this is leading to the confusion matrix?

> *The logistic regression's accuracy is just above the do-nothing model's, which finds no failed drive; the logistic regression finds 43 of 76. It is leading to the confusion matrix?*

Yes. 96.20% is simply the share of healthy test drives, $1 - \pi$ with = 76 / 2,000 = 0.038, so the do-nothing model gets it for free. And the table already hides a second number: 1,924 - 1,911 = 13 healthy drives were flagged by the logistic regression, false alarms, the price of its 43. Two totals cannot show which errors were made; four counts can.

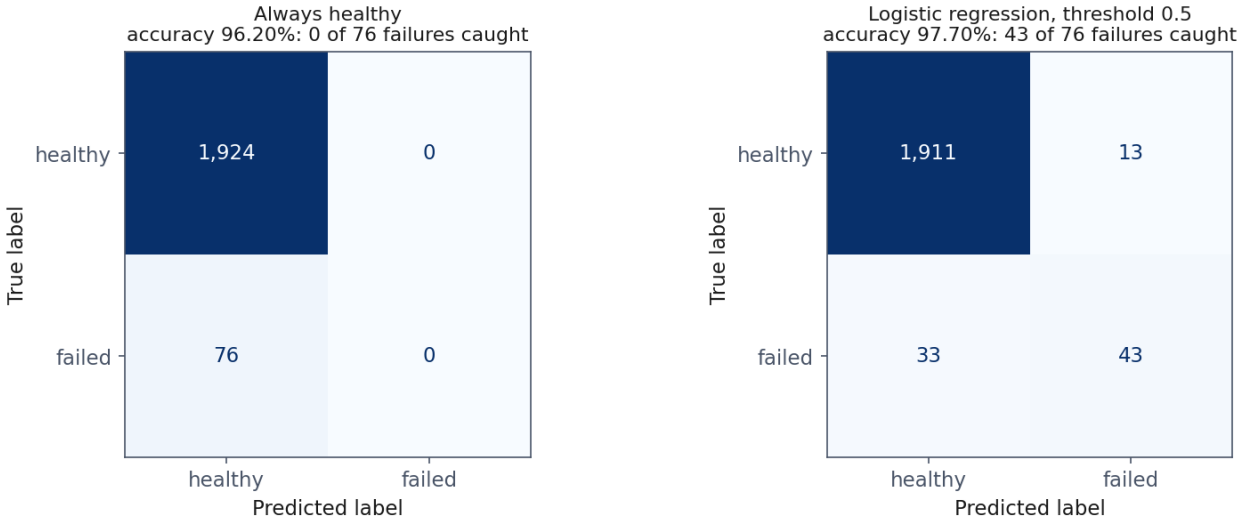
Slide 34 — Why everyone makes this mistake once

Accuracy is not a naive metric. On a balanced problem whose two errors cost the same, it is the right summary. It stops being informative when one class is rare: if a share π of cases is positive, the always-negative classifier scores $1 - \pi$, 96.2% here and 99.9% in a fraud problem with one case in a thousand.

Question — classes are almost always imbalanced, so accuracy is useless?

Right in substance, with two refinements. The slide says the **interesting** class is almost always the rare one, not that every dataset is imbalanced. And "useless" overshoots: accuracy is a number with a baseline. Read against $1 - \pi$ (96.20% here), the model's 97.70% does say something. What accuracy cannot do, even on a balanced problem, is tell a miss from a false alarm, and here a miss costs 18.6 times more.

Slide 35 — Four numbers instead of one



Truth in rows, prediction in columns, scikit-learn's convention. The bottom-left cell is the failures called healthy, the one that costs money.

Read the bottom-left cell first when misses are expensive: 33 drives failed and were called healthy. The top-right cell, 13, is the price of the 43 caught. Half the literature uses the transposed layout, so always check which way round a confusion matrix is drawn before reading it.

Question — 33 missed: so there is room to improve the model?

> *The model found 43 failures but missed 33, so there is room to improve it; the do-nothing model finds none because it returns the most frequent class.*

The reading is right (the do-nothing model is `DummyClassifier(strategy="most_frequent")`, and its whole right-hand column is 0). But "room to improve" is not necessarily the model. The first lever is the threshold: with the same probabilities, a lower threshold catches more failures at the price of more false alarms (slides 42 to 44, and the threshold derived from the costs on slide 55). The threshold trades one error for the other; only a better model, one that ranks the drives better, reduces both.

Slide 36 — The four outcomes, and what they cost

The four outcomes have names: **true positive (TP)**, a failure caught; **false positive (FP)**, a healthy drive flagged; **false negative (FN)**, a failure missed; **true negative (TN)**, a healthy drive left alone. The slide prices them. A false positive, a healthy drive replaced, costs a technician's hour, €140; a false negative, a drive that dies in service, costs the callout, the rebuild and the risk, €2,600. The two right calls count as €0, because each figure is what an error adds over the right decision. The costs are assumptions supplied from outside the data, not something the model measures, and slides 53 to 55 turn them into a threshold.

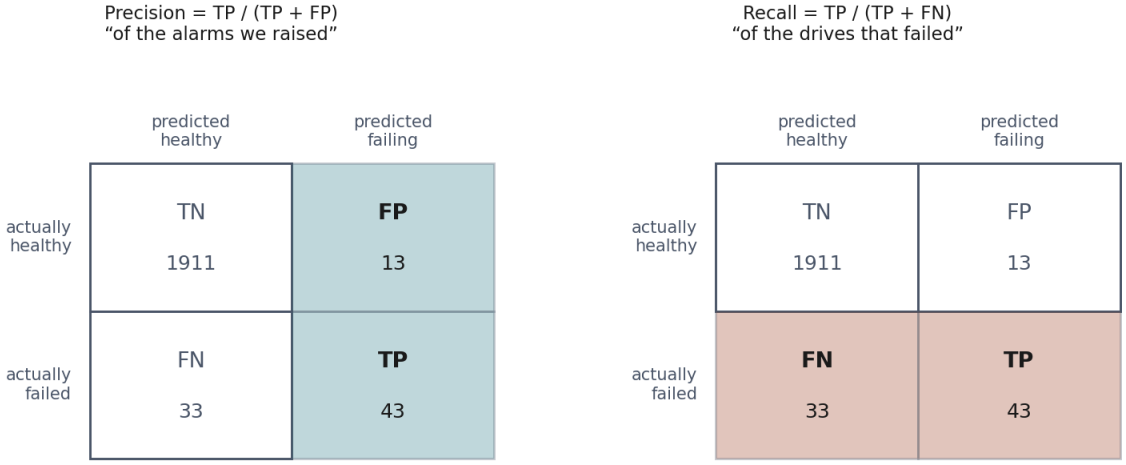
Question — the costs depend on the application, not on the model?

Right: they are assumptions supplied from outside the data. The model gives the probabilities, the costs give the threshold. And the two errors are not the same size, which no single-number metric knows: accuracy and F1 both treat them as equal.

Question — and does a true positive cost nothing?

Replacing a drive that really was failing costs a technician's hour, but a failing drive has to be replaced anyway, so the model did not cause that cost. Each figure is what an error **adds** over the right decision, and correct decisions add nothing. With all four outcomes priced, the threshold of slide 55 becomes $t^* = (C_{FP} - C_{TN})/((C_{FP} - C_{TN}) + (C_{FN} - C_{TP}))$, in which only differences enter.

Slide 37 — Precision and recall: look at the denominator



Both metrics put the true positives on top. Precision divides by the column, the drives flagged; recall by the row, the drives that failed.

Precision and recall share a numerator, the true positives, and differ only in what they divide by. Precision divides by the drives flagged, TP + FP: of the alarms, how many were real. Recall divides by the drives that failed, TP + FN: of the failures, how many were found. The hook: **precision is about the alarms, recall is about the failures**. Precision answers the technician ("how much of my time are you wasting?"), recall the operations manager ("how exposed am I still?").

Question — to maximise recall make FN zero, to maximise precision make FP zero, both by moving the threshold?

> *TP is always on top; to maximise recall zero the false negatives, to maximise precision zero the false positives; both by moving the threshold. Correct?*

Correct, with two additions. Each extreme is trivial, which is why neither alone is a goal: flag everything and recall is 1 while precision is 0.038; raise the threshold to 0.90 and precision is 1.000 while recall falls to 14 of 76, 0.184. Hence F1 (slide 39) and, better, the costs (slides 53 to 55). And the threshold only trades one error for the other along a fixed model; reducing both takes a model

that ranks the drives better, which moves the whole precision-recall curve. Recall can only fall as the threshold rises; precision usually rises but is not guaranteed to, because the true positives fall too.

Slide 38 — Our model, in those terms

The slide puts the model's four counts, from the confusion matrix of slide 35, into the two metrics of slide 37 and a third. Each line read as a sentence with its counts: of the 56 drives flagged, 43 failed (precision 0.768); of the 76 that failed, 43 were found (recall 0.566); of the 1,924 healthy drives, 1,911 were left alone (specificity 0.993).

Question — specificity is the recall of the negative class, and one minus it is the false positive rate?

Yes. The **false positive rate (FPR)** is FP / (TN + FP) = 13 / 1,924 = 0.007, and it becomes the horizontal axis of the ROC curve on slide 46.

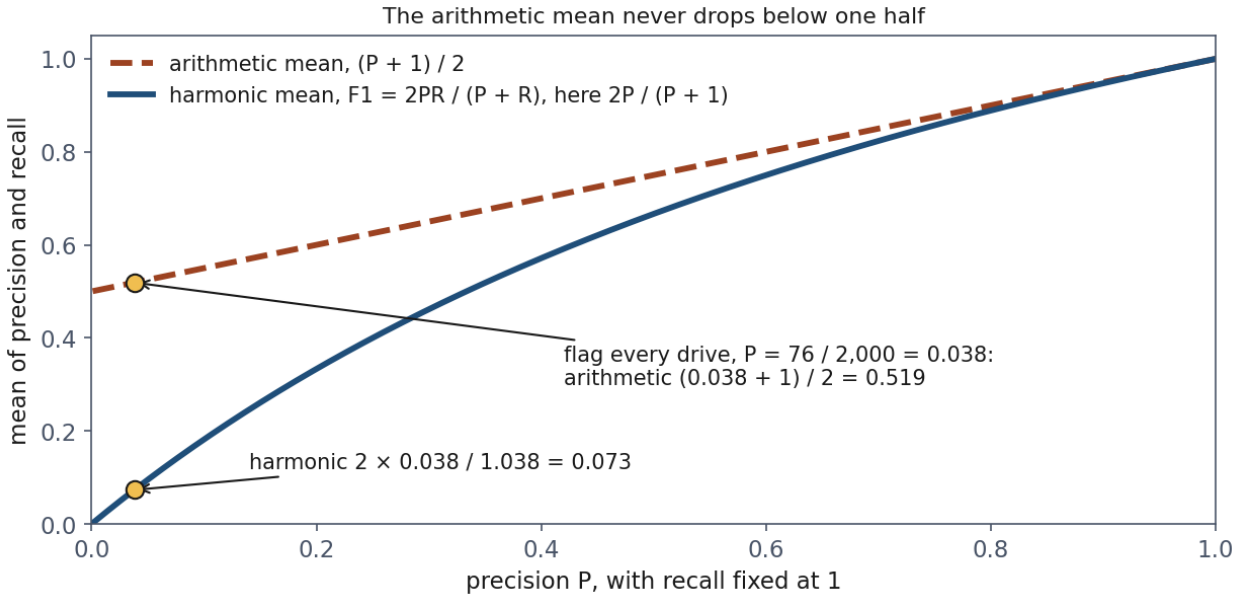
Worked example — twenty drives, four of which fail

A model flags 5 of 20 test drives; 3 of the 5 really fail, and 1 failure is missed: TP = 3, FP = 2, FN = 1, TN = 14.

metric	formula	value
accuracy	(TP + TN) ÷ 20 = (3 + 14) ÷ 20	0.85
always "healthy"	(FP + TN) ÷ 20 = 16 ÷ 20	0.80
precision	TP ÷ (TP + FP) = 3 ÷ 5	0.60
recall	TP ÷ (TP + FN) = 3 ÷ 4	0.75
F1	2TP ÷ (2TP + FP + FN) = 6 ÷ 9	0.667
specificity	TN ÷ (TN + FP) = 14 ÷ 16	0.875
false positive rate	1 − specificity	0.125

The model beats the do-nothing baseline by 5 points of accuracy, and the other rows say what those points are made of: three failures caught out of four, at the price of two needless replacements. F1 from the counts and from $2PR/(P+R)$ give the same 0.667.

Slide 39 — F1 = 2PR ÷ (P + R): why the harmonic mean



Recall fixed at 1, precision along the axis. The arithmetic mean (dashed) never drops below one half; the harmonic mean, F1, follows the weaker of the two.

Precision and recall are two numbers; F1 combines them into one, with the harmonic mean, $F_1 = 2PR/(P + R)$, instead of the ordinary average. The title gives the formula; in counts it is $2TP/(2TP + FP + FN)$, and our model's value is on the next slide. Why the figure holds recall at 1: it is the model that flags every drive, recall 76 ÷ 76 = 1 and precision 76 ÷ 2,000 = 0.038, the simplest case in which one of the two numbers is perfect and the other is useless. The figure shows why: with recall fixed at 1 and precision falling, the arithmetic mean never drops below one half, while F1 follows the weaker of the two down to 0. For our model, precision 0.768 and recall 0.566 give F1 = 0.652.

Question — why the harmonic mean, and are there other reasons?

> *For $R = 1$ and $P = 0$ the arithmetic mean is about 0.5 and says nothing, F1 gives 0.073. The harmonic mean's formula is not shown. Other reasons not to use the arithmetic mean? And F2: is recall multiplied by 2?*

First a precision on the numbers: 0.519 and 0.073 belong to the flag-everything model, whose precision is 0.038, not 0; at precision exactly 0, F1 is 0. The formula is $F_1 = 2PR/(P + R)$, the reciprocal of the average of the reciprocals. Three reasons for it:

1. **The weaker side dominates.** A small number has a large reciprocal, so a perfect score on one axis cannot buy a pass on the other.
2. **It is still a ratio of drives.** $F_1 = 2TP/(2TP + FP + FN)$: true positives over the average of the two denominators. Ours: $86/(86 + 13 + 33) = 0.652$.
3. **The arithmetic mean can be gamed**: flagging everything earns at least one half for free.

F_2 does not multiply recall by 2. In $F_\beta = (1 + \beta^2)PR/(\beta^2 P + R)$ it is recall's reciprocal that gets the weight $\beta^2 = 4$, against 1 for precision's: $F_2 = 5PR/(4P + R) = 0.597$ for our model, below F1 because recall is our weak side.

Question — a miss costs 19 times a false alarm: which matches that?

None. F_β expresses a preference between precision and recall, not the costs: nothing turns 2,600 / 140 = 18.6 into a β, and F_2 is a common descriptive choice, not a cost model. To act on the costs, derive the threshold from them (slide 55).

Slide 40 — Read the minority class, not the average

F1 can be computed for each class in turn, and `classification_report` prints both rows plus two averages over the classes. Healthy scores 0.988 on 1,924 drives, failed 0.652 on 76. The **macro** average gives each class the same say, $(0.988 + 0.652) \div 2 = 0.820$; the **weighted** average weighs each by its number of drives, $(1{,}924 \times 0.988 + 76 \times 0.652) \div 2{,}000 = 0.975$, which is the healthy class talking. On imbalanced data the weighted average reads like accuracy and hides the minority: read the minority class's own row first.

Question — F1 for the negative class, and macro and weighted averages: new metrics?

F1 can be computed with either class as the positive one: for the healthy class, $2 \times 1{,}911 / (2 \times 1{,}911 + 13 + 33) = 0.988$. Macro and weighted are not new metrics but two **averaging rules** over classes, applied by `classification_report` to every column (precision, recall, F1). Macro gives each class the same say, $(0.988 + 0.652)/2 = 0.820$; weighted gives each its share of the test set, the **support**, $(1{,}924 \times 0.988 + 76 \times 0.652) / 2{,}000 = 0.975$, which is the healthy class talking and reads close to accuracy.

Question — why is an arithmetic mean fine here, one slide after F1 rejected it?

Slide 39 combined two different quantities of one class, where the weaker had to dominate. Here it is the same metric on two classes, and the point is to give the rare class an equal voice. Which to report: the minority class directly, and the macro average when the classes should count equally; if you quote the weighted average, say so.

Slide 41 — Notebook 2, live

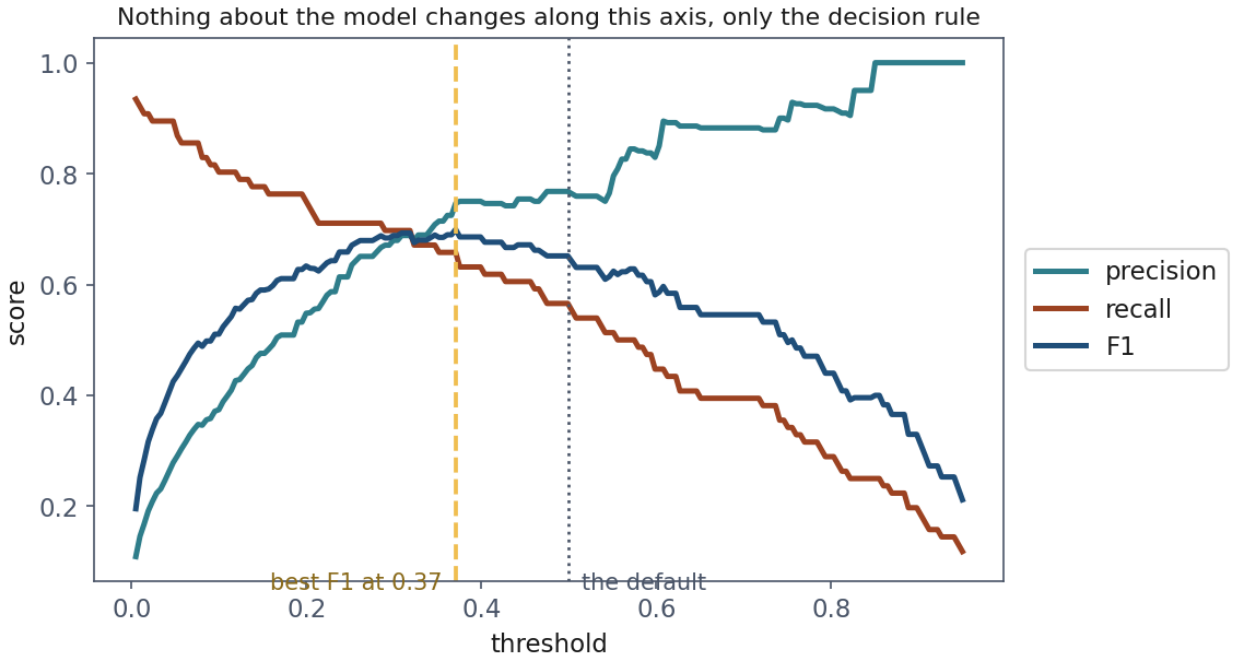
Notebook 2 builds the confusion matrix by hand before calling `confusion_matrix`, then the metrics, the threshold sweep and the precision-recall curve. Building the four counts by hand takes two minutes and is what makes the jargon stop being jargon.

Question — does the opening say the always-healthy model wins?

It says it looks excellent: 96.20%, only $97.70 - 96.20 = 1.5$ points below the logistic regression, which catches 43 of the 76 failures. "Looks excellent and catches nothing" is the point; if accuracy were the only measure, nobody would notice.

Part VI — The threshold

Slide 42 — The threshold is a choice nobody made



Nothing about the model changes along the horizontal axis: same coefficients, same probabilities. Only the line between "leave it" and "replace it" moves.

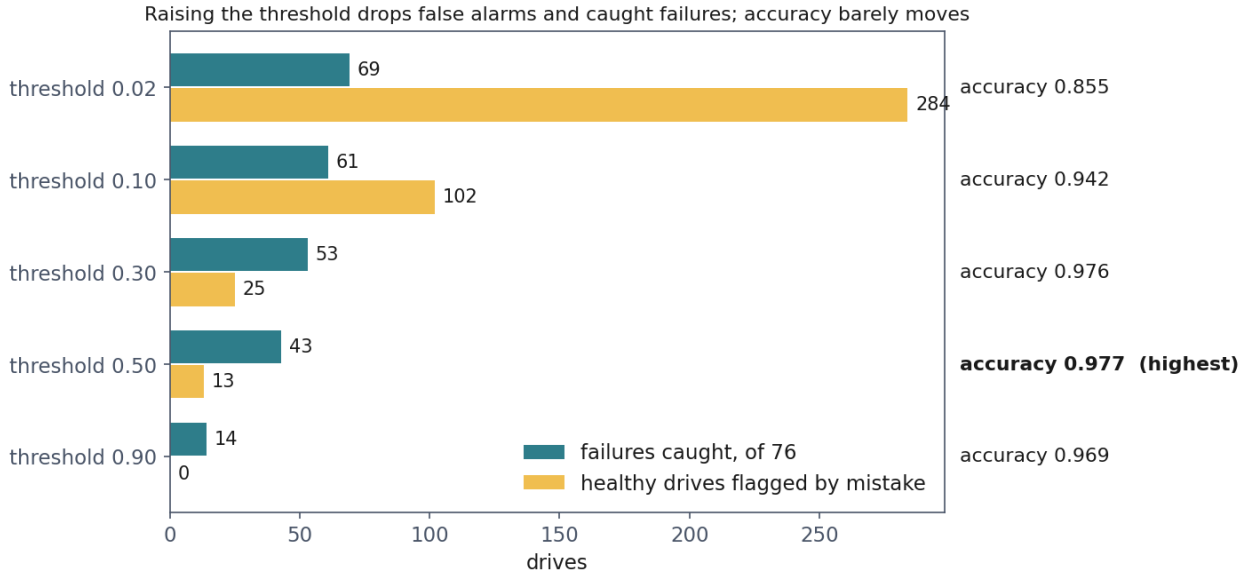
The figure sweeps the threshold from 0 to 1 on one model, with the same probabilities throughout, and draws precision, recall and F1 at each value. Recall can only fall as the threshold rises, because fewer drives are flagged; precision mostly rises. Nothing about the model changes along the axis, only where the line between "leave it" and "replace it" is drawn, and 0.5 is no more natural a place for it than any other: it is the default of `predict()`, which nobody chose for this problem.

Question — so 0.5 is only predict()'s default, and a precision of 77% means little without its threshold?

> *0.5 is only the default of predict(); the best threshold depends on the costs of the two errors, derived later; 77% precision says little without the threshold and why it was chosen, and usually nobody asks.*

Correct, with one condition: 0.5 is right when the two errors cost the same **and** the probabilities are calibrated (among drives given 0.3, about 30% really fail; slide 49 names the property). On this test set F1 peaks at 0.30 (0.688), accuracy at 0.50 (0.977, missing 33 failures), and the threshold from the costs is 0.051. Without costs, other rules can be defended, such as the best F1, or a capacity: "the technicians can check so many drives a week", so flag the top of the ranking.

Slide 43 — What the sweep actually costs



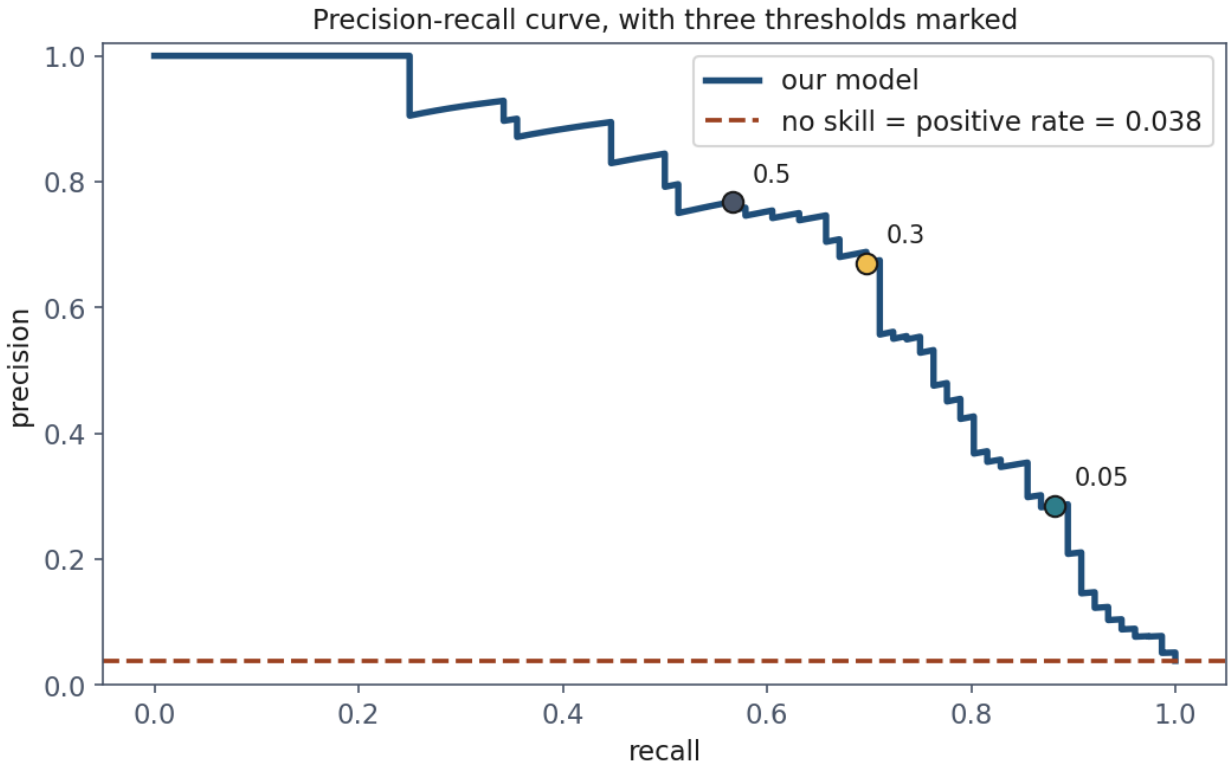
Five thresholds as bars: failures caught (teal) and healthy drives flagged (gold). On the right, the accuracy each threshold scores.

The same sweep as counts, which is what the people running the fleet see: at each of five thresholds, the failures caught and the healthy drives flagged, with the accuracy each threshold scores beside them. Lowering the threshold buys failures with false alarms. The column to read is the accuracy: it barely moves across the range, and it peaks at the threshold that misses the most failures of any sensible choice, which is why accuracy cannot be the criterion for choosing.

Question — low threshold, almost every failure caught and many false alarms; at 0.9 no false alarms?

Correct: at 0.02 the model catches 69 of 76 failures and flags 284 healthy drives; at 0.90 it flags 14 drives, all failures, precision 1.000, and misses 62. The argument is in the right-hand column. From 0.10 to 0.90 accuracy moves only between 0.942 and 0.977, and it peaks at 0.5, the threshold that misses 33 failures. Most alarms a higher threshold drops are false ones, so accuracy rises with each drop even when caught failures are dropped too: from 0.30 to 0.50 the model loses 12 false alarms and 10 caught failures, two right answers more in all. Tune on accuracy and it will quietly recommend catching fewer failures.

Slide 44 — The precision-recall curve



Every threshold at once. The dashed line at 0.038 is a model that ranks at random.

The precision-recall curve draws every threshold at once: recall on the horizontal axis, precision on the vertical, one point per threshold, from the strictest on the left to "flag everything" on the right. The dashed line is what a random ranking would give: a precision equal to the share of failures, 0.038, at every recall. The area under the curve, the average precision (AP), is 0.717 for our model against 0.038 for chance (notebook 2).

Question — how is the baseline computed?

> *Precision always 0.038 = 76 / 2,000, whatever the recall?*

Yes, and the reason is that **a random selection keeps the proportions** of what it is drawn from. Picture an urn with 2,000 balls, 76 of them red, the drives that fail: 3.8%. A model that flags drives at random is drawing balls blind, and whatever handful it draws holds, on average, red balls in the same proportion as the urn. Nothing makes the red ones more likely to be drawn.

Flag k drives at random. The expected number of failures among them is k × 76 ÷ 2,000 = k × 0.038, so precision = failures ÷ flagged = (k × 0.038) ÷ k = 0.038: k cancels. Recall = failures ÷ 76 = (k × 0.038) ÷ 76 = k ÷ 2,000, the share of drives flagged.

drives flagged, k	failures expected = $k \times 0.038$	precision = failures ÷ k	recall = failures ÷ 76
500	19	19 ÷ 500 = 0.038	19 ÷ 76 = 0.25
1,000	38	38 ÷ 1,000 = 0.038	38 ÷ 76 = 0.50

Two words in that reasoning matter. 0.038 is a **share** of the drives flagged, not a number of drives: of 500 flagged, 19 fail. And it holds **on average**: one random draw of 500 drives may hold 15 failures or 23, and flagging only a handful gives very uneven results. Flagging more moves recall, never precision. So "no skill" is a flat line at the positive rate, not at 0.5, which holds only on a balanced problem. Our curve stays far above it; its area, the **average precision (AP)**, is 0.717 against 0.038.

Part VII — Ranking: the ROC curve and the area under it

Slide 45 — Comparing models without choosing a threshold

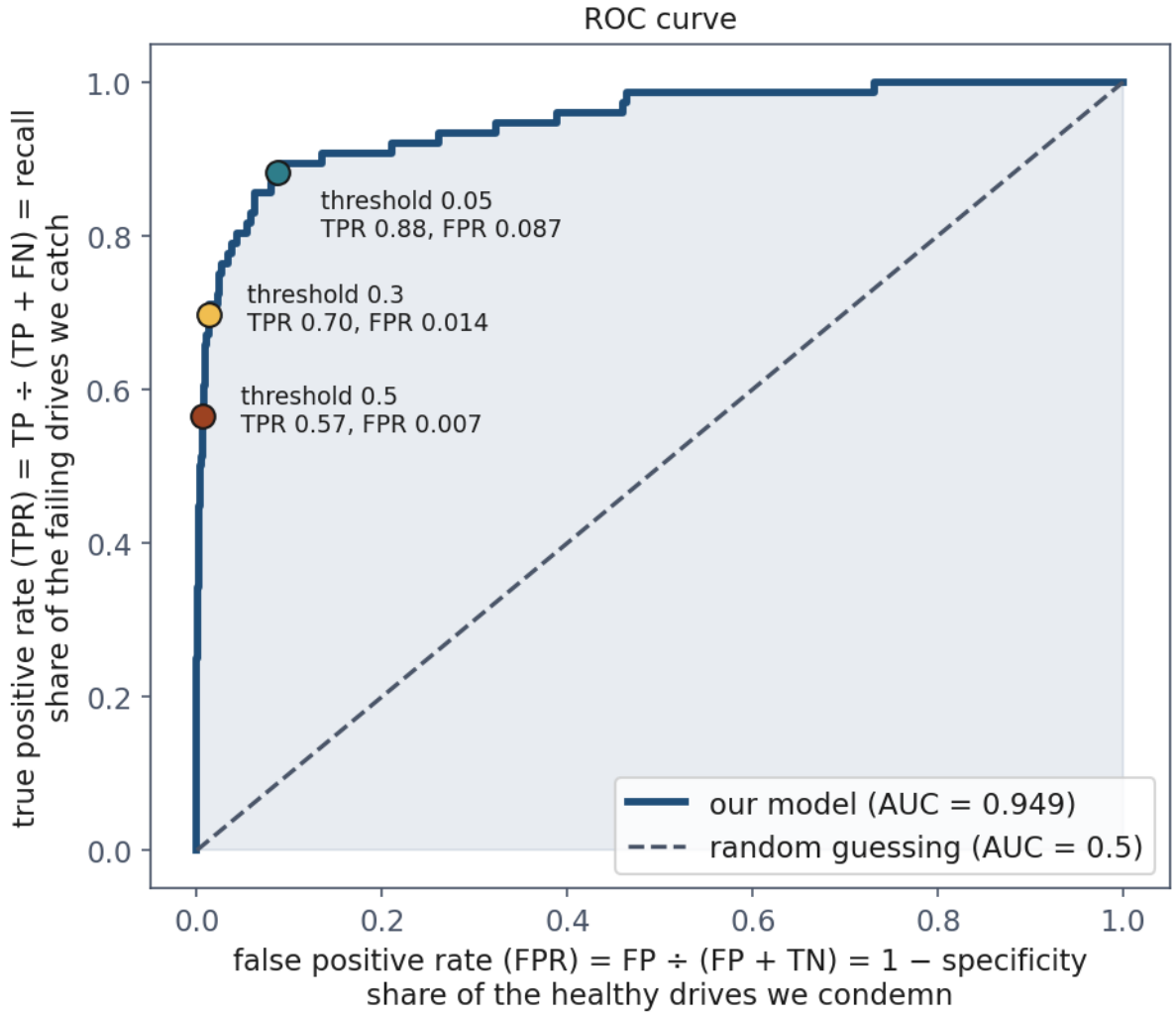
So far every metric needed a threshold. This slide changes the question: instead of judging the model at one cut, judge its **ranking**, the 2,000 test drives sorted by predicted probability, riskiest first. A threshold is just a cut in that list, everything above it flagged, so scoring the ranking means scoring every cut at once. The next slides do it: the ROC curve (slide 46), the area under it (slide 47) and what that area is blind to (slide 49).

Question — a new idea: judge the model by its ranking, not at one threshold?

> *So far models were compared at a threshold; now the idea is to evaluate a model without the threshold, only from the ranking. Right?*

Right, with one link made explicit. The **ranking** is the 2,000 test drives sorted by predicted probability, riskiest first. A threshold is a cut in that list: everything above the cut is flagged. So a threshold-free measure does not throw the threshold away; it looks at every cut of the same list at once, and the precision-recall curve was already a first step. Two conditions: only the order counts, not the values of p, so a ranking measure is blind to calibration (slide 49); and in use, one cut still has to be chosen (slides 53 to 55).

Slide 46 — The ROC curve



True positive rate (recall) against false positive rate, over every threshold. The diagonal is a random ranking; the top-left corner is perfection.

The receiver operating characteristic (ROC) curve walks down the ranking and, at every cut, plots the share of failures caught, the true positive rate (TPR, the same as recall), against the share of healthy drives flagged, the false positive rate (FPR). It starts at (0, 0), flag nothing, and ends at (1, 1), flag everything. A random ranking follows the diagonal; a perfect one goes straight up to (0, 1) and then across. Our model's curve rises steeply at the start: the top of its ranking is mostly failures.

Question — a steep start means a good model?

> *False positive rate on x, true positive rate on y, over every threshold. Bottom left is threshold 1, top right is flag everything. The steeper the curve at the start, the more failures found without many false alarms. Right?*

Right. Walk down the ranking one drive at a time: a failing drive moves the curve up by 1/76, a healthy one moves it right by 1/1,924. A steep start means the top of the ranking is failures. The

ideal goes straight up to (0, 1), then across; every curve ends at (1, 1), so steep at the start means flat at the end, which is what the area measures. One point to check: threshold 0.5 gives a **true positive rate (TPR)** of $43/76 = 0.57$ and an FPR of 13 / 1,924 = 0.007.

The sweep is the same as the precision-recall curve's; the reading is not. The precision-recall curve's ideal is the top-right corner, and precision divides by the alarms raised. The ROC's ideal is the top-left corner, and its FPR divides by the 1,924 healthy drives, so its random baseline is the diagonal whatever the class balance. That denominator is why the ROC can hide imbalance (slides 50 and 51).

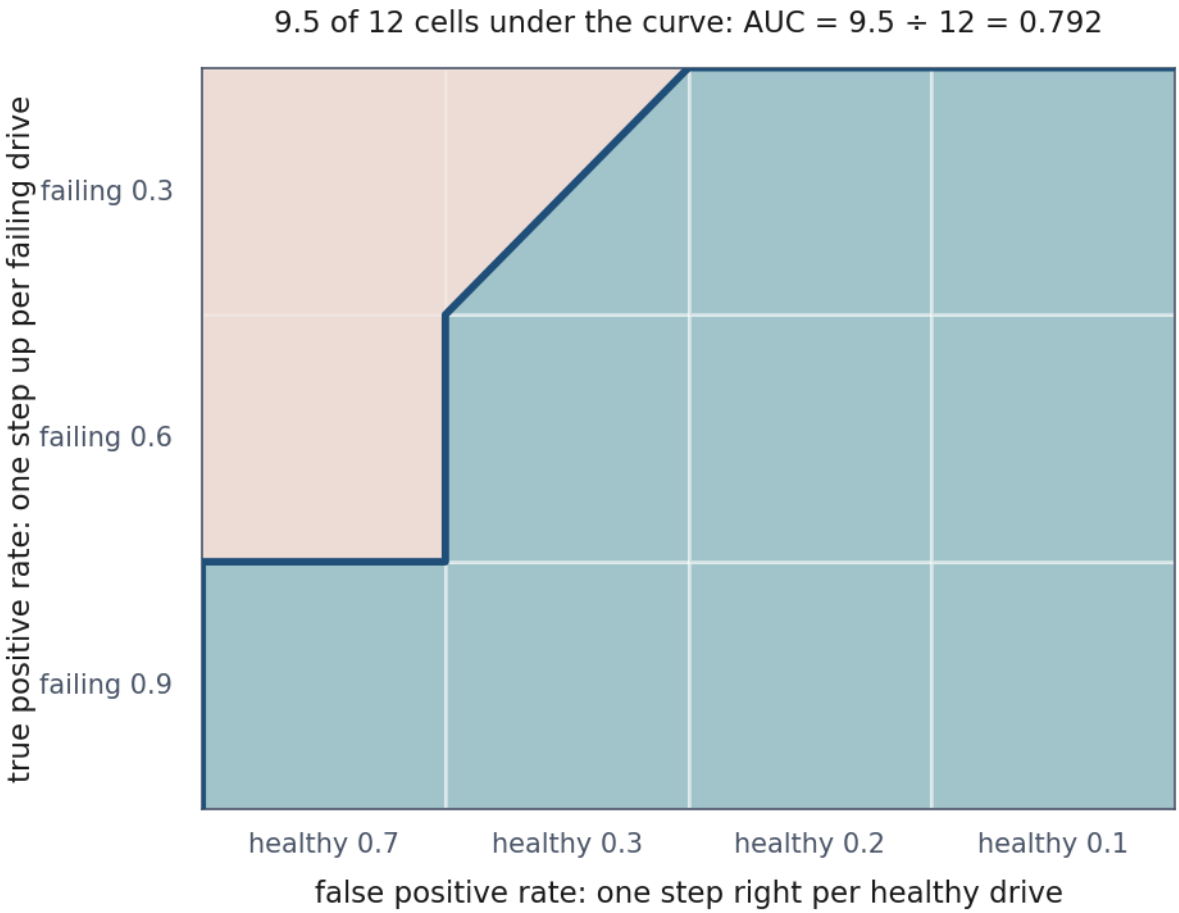
Question — why is recall suddenly called the true positive rate?

Same quantity, $TP/(TP+FN)$; ROC analysis came from radar engineering, and that field's names stuck.

Slide 47 — What the area means

The area under the ROC curve (AUC) has a reading that needs no curve: it is the probability that a failing drive picked at random scores above a healthy drive picked at random. Ours is 0.949 (`roc_auc_score`, notebook 3), and notebook 3 checks it by brute force: in 200,000 random (failing, healthy) pairs, the failing drive scores higher 94.9% of the time. 0.5 is a random ranking, 1 a perfect one.

Why that probability is an **area**: the ROC curve is a staircase on a grid. Walking down the ranking, each failing drive moves the curve up by 1/76 and each healthy drive moves it right by 1/1,924, so the unit square is cut into $76 \times 1{,}924$ cells, one per (failing, healthy) pair. In a healthy drive's column, the curve crosses at the height reached so far, the share of failing drives scored above it; so the cells under the curve are exactly the pairs in which the failing drive wins. Adding the columns, the area is the share of pairs in the right order. A tie is a diagonal step, which cuts its cell in half.



The seven drives of the worked example below, on their grid of 3 × 4 = 12 cells. Look at the teal cells under the curve: in each, the failing drive of its row outscores the healthy drive of its column; the tie at 0.3 is cut by the diagonal and counts half. 9.5 of 12 cells: AUC = 0.792.

Question — is the AUC a probability or a count?

> *Pick at random one failed and one healthy drive; the AUC is the probability, or the number of times, that the model scores the failed one higher? So a model can have a high AUC without being calibrated?*

Both, which are the same number. The test set holds 76 × 1,924 = 146,224 (failing, healthy) pairs. The exact **area under the curve (AUC)** is the share of them in which the failing drive scores higher, a tie counting half: 7,409 pairs are in the wrong order, so 1 - 7,409 / 146,224 = 0.9493. Notebook 3 draws 200,000 random pairs instead and gets 0.9488; the difference is sampling noise. And yes: ranking and calibration are separate properties, as slide 49 shows in both directions.

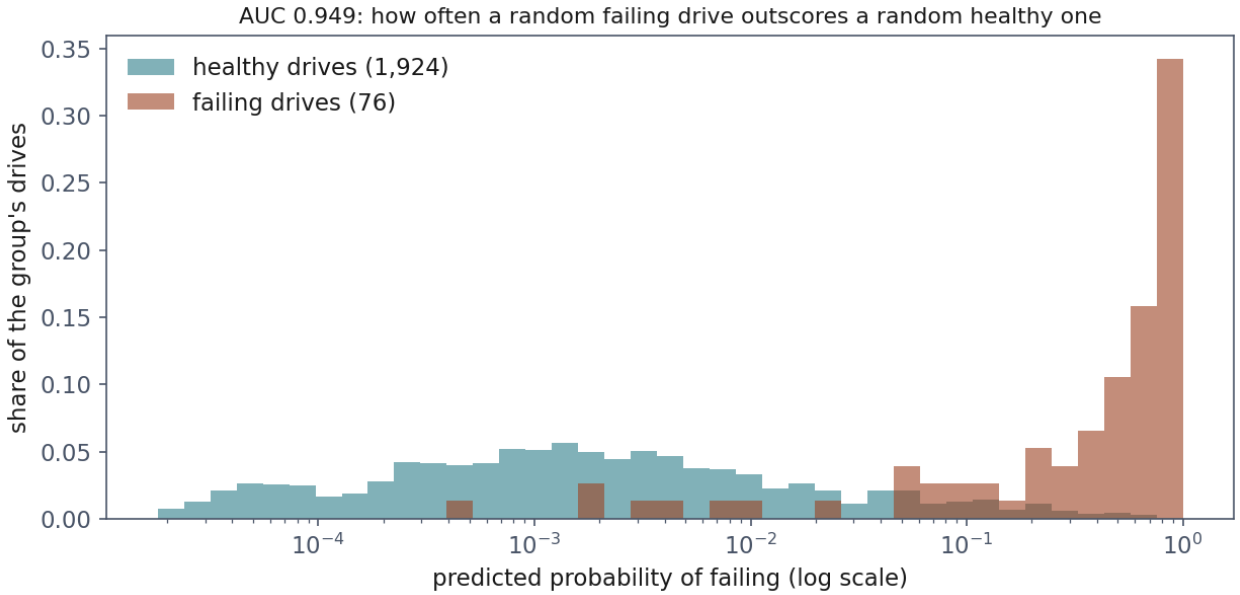
Worked example — the AUC of seven drives, by counting pairs

Three failing drives scored 0.9, 0.6, 0.3; four healthy ones scored 0.7, 0.3, 0.2, 0.1. There are $3 \times 4 = 12$ (failing, healthy) pairs. For each failing drive, count the healthy drives it outscores, a tie counting half:

failing drive's score	healthy drives it outscores	pairs won, a tie counting ½
0.9	0.7, 0.3, 0.2, 0.1	4.0 of 4
0.6	0.3, 0.2, 0.1	3.0 of 4
0.3	0.2, 0.1, half of 0.3 (tie)	2.5 of 4

AUC = (4.0 + 3.0 + 2.5) ÷ 12 = **0.792**. Drawing the ROC curve threshold by threshold and measuring its area gives the same 0.792: the probability reading and the geometric one are one number. The drive at 0.6 loses one pair, to the healthy drive at 0.7; the drive at 0.3 loses one and ties one.

Slide 48 — The scores behind the AUC



Scores of the healthy (teal) and failed (rust) test drives, each group scaled to add up to 1. The horizontal axis is logarithmic.

The figure shows the predicted probabilities behind the AUC: the healthy test drives in teal, the failed ones in rust, each group scaled to add up to 1, on a logarithmic axis. The two humps overlap only in part: most failures score far above most healthy drives, which is what the 0.949 measures. The wrong pairs come from the overlap, a failure scored to the left of healthy drives, and the questions below count them.

Question — is the height the number of drives at that probability?

No: the share of each group's drives, each colour adding up to 1. As counts, the 76 rust drives would be invisible next to the 1,924 teal ones. Converted back: the tallest rust bar, 0.342, is 26 drives; the tallest teal bar, 0.056, is 108. The low bar holds four times as many drives.

Question — the failed drives to the left of healthy ones are what lowers the AUC?

Yes, and they can be counted. A failing drive costs one wrong pair for every healthy drive scored above it, so a failure far to the left costs a lot: the lowest-scoring one ($p = 0.0004$) is outscored by 1,408 healthy drives, a fifth of all the wrong pairs by itself. The 5 worst failures make 61% of them; 19 failures score above every healthy drive and cost nothing.

Slide 49 — What AUC does not tell you

The AUC sees only the order of the drives, and the slide proves it with an experiment. Divide every predicted probability by 10: the order is unchanged, so the AUC is still 0.949, but every decision changes, because at threshold 0.5 the model now flags none of the 56 drives it flagged before. What broke is **calibration**: a model is calibrated when a predicted 0.7 comes true about 70% of the time. Ranking and calibration are separate properties, and the cost threshold of slide 55 needs the second.

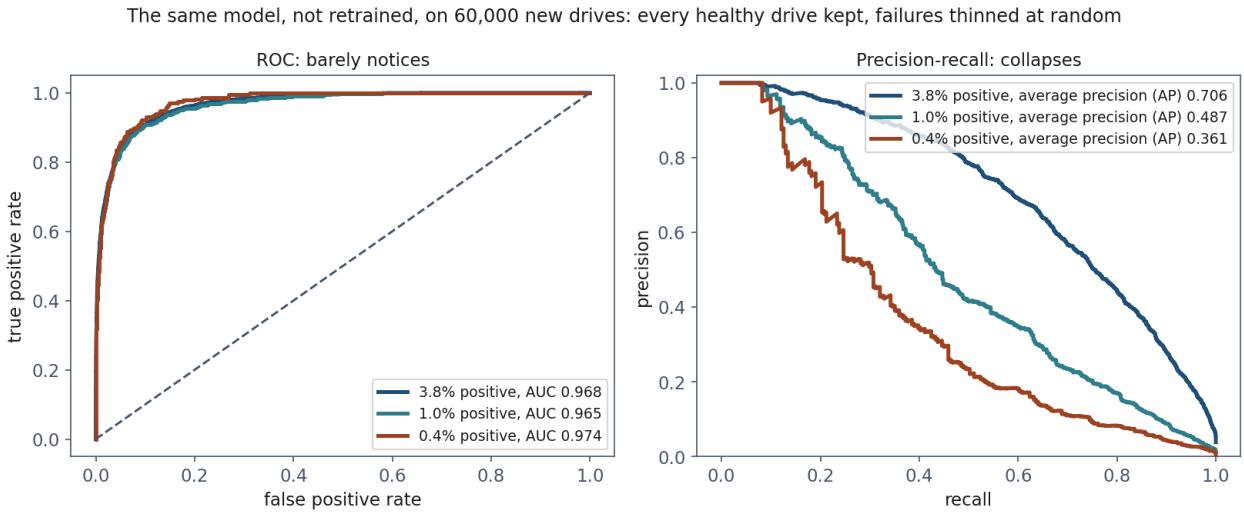
Question — divide by ten, same AUC: discrimination is not calibration?

Right, with its condition: the AUC is unchanged by any change of the scores that keeps their order (any strictly increasing one); a change that reverses the order turns it into $1 - \text{AUC}$. Divided by ten, the largest score is 0.100, so at threshold 0.5 the model flags none of the 56 drives it flagged before, recall 0, AUC still 0.949. The converse holds too: give every drive 0.038, the fleet's failure rate. On average that is right, calibrated, and it ranks nothing: AUC 0.5.

Question — an AUC of 0.95 means the probabilities are reliable?

No: the AUC sees only the order. Whether "0.7" comes true 70% of the time is **calibration**, a separate property, and it is the one the cost threshold of slide 55 depends on.

Slide 50 — Where ROC misleads



The same model on three fleets with failures thinned from 3.8% to 0.4%. Left, three ROC curves on top of each other; right, three precision-recall curves far apart.

The slide tests the AUC where it is weakest. The same model, not retrained, is applied to three fleets in which the failures are thinned at random, from 3.8% to 0.4%, while every healthy drive is kept. On the left the three ROC curves lie on top of each other; on the right the three precision-recall curves fall far apart, the average precision from 0.706 to 0.361. The ROC curve does not see what rarer failures do to the queue of alarms a technician has to work through.

Question — the ROC curves barely move while precision and AP fall?

> *Same model, not retrained, a fleet of 60,000 drives, failures brought from 3.8% to 0.4%. The ROC curves barely differ, the right panel is worse. Right?*

Right. Precisely: 60,000 drives are generated, all 57,575 healthy ones are kept, and only failures are discarded, so the three fleets hold 59,849, 58,157 and 57,806 drives. The AUC reads 0.968, 0.965, 0.974; the average precision falls from 0.706 to 0.361. Why the ROC does not move: both its axes are shares inside one class. Recall is a share of the failures, and thinning them at random leaves it where it was (0.50, 0.45, 0.46 at threshold 0.5); the FPR is a share of the healthy drives, which are untouched.

Question — why is the AUC 0.968 here and 0.949 on slide 47?

Different drives: 0.949 rests on the 76 failures of the 2,000-drive test set, 0.968 on a fleet thirty times larger. Both estimate the same ranking ability.

Slide 51 — Why the two curves disagree

First, why the slide talks about the false positive rate at all. The two curves of slide 50 share one axis, recall: the ROC's vertical axis is the precision-recall curve's horizontal one. They differ in one axis each, the false positive rate on the ROC and precision on the other. So the slide takes the three in turn. **Recall**, TP ÷ (TP + FN), has failures above and below, caught ones and missed ones;

thinning the failures at random removes both in the same proportion, so it stays put: 0.50, 0.45, 0.46, noise around one value. The **false positive rate** divides by all the healthy drives, which the thinning never touched: 304 false alarms out of 57,575, 0.0053, in all three fleets. Precision divides by the alarms raised: the 304 false alarms stay, while the true positives fall with the failures, from 1,127 to 106, so the false alarms' share of the alarms grows from 0.21 to 0.74. Each of the ROC's two axes is a share inside one class; precision mixes the two classes, which is why only precision feels the imbalance.

Question — why does precision fall faster than recall when failures are removed?

Recall does not fall at all: it moves only by noise. Its numerator and denominator are both failures, so thinning them at random leaves the ratio where it was. Precision has failures on top but healthy drives in its denominator, through the false positives, and those are untouched: TP falls with the failures, FP stays at 304, and precision falls from 1,127 ÷ (1,127 + 304) = 0.79 to 106 ÷ (106 + 304) = 0.26. A small wording trap on the way: the false negatives are not "the false ones"; they are failures, the ones the model called healthy, which is exactly why they thin together with the true positives.

Question — FPR unchanged, recall unchanged, so the ROC is fixed; and precision falls because the false positives grow?

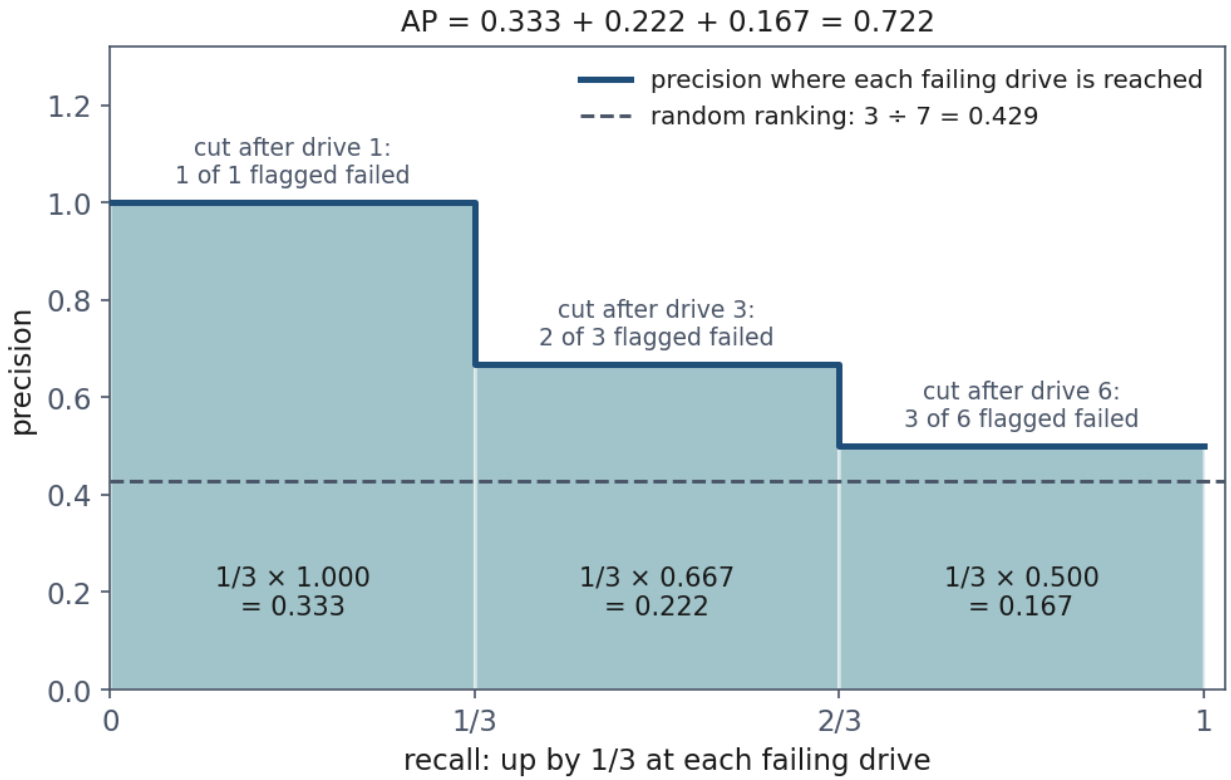
> *FPR keeps all healthy drives in the denominator, so thinning the positives leaves it unchanged; recall is unchanged too; so the ROC does not change. Precision falls because FP grows. Right?*

Right on the ROC, imprecise on the last step. The false positives do **not** grow: they are 304 in all three fleets, because the healthy drives are the same ones. What falls is the true positives, 1,127 → 261 → 106, with the failures. What grows is the false alarms' **share** of the alarms: 304 / (1,127 + 304) = 0.21 at 3.8% and 304/(106 + 304) = 0.74 at 0.4%, which is 1 − precision. In one formula, with π the positive rate: precision $= R\pi/(R\pi + \mathrm{FPR}(1-\pi))$. R and FPR are the ROC's axes and stay put; only π moves. "The false alarms take over" sounds like "there are more of them", which is why the slip is natural.

So the ROC measures false alarms against a population nobody experiences. The technician does not see the 57,000 drives left alone; they see the alarm queue.

Slide 52 — The rule: report both

The rule that follows: on imbalanced data, report the precision-recall curve beside the AUC, not instead of it. The two answer different questions, "are failures ranked above healthy drives?" and "how many alarms are real?". The precision-recall curve's summary is the **average precision (AP)**, and the idea comes before the formula: every time the ranking reaches a failing drive, take the precision at that point, how clean the queue of alarms is at that moment, and average those precisions over the failures. As a formula, $\sum_k (R_k - R_{k-1})P_k$: recall rises only at a failing drive, by 1/76 each time, so each precision is weighted by the same step and the weighted sum is the plain average.



The seven-drive ranking of the worked example below. Look at the three rectangles, one per failing drive: each is as wide as the recall it adds, 1/3, and as tall as the precision where that drive is reached; their areas add up to the AP, 0.722. Ours is 0.717 (notebook 2), read against a random ranking's 76 ÷ 2,000 = 0.038, never against 0.5.

Question — "its area": the area of the precision-recall curve?

Yes. The two measures answer two different questions: the AUC, "does the model put the failures above the healthy drives?"; the precision-recall curve, "of the alarms it raises, how many are real?". On a balanced problem the answers travel together; on an imbalanced one the second can collapse while the first stands still. The average precision is computed as a step sum, $\sum_k (R_k - R_{k-1}) P_k$: walk down the thresholds, and each time recall rises by a step, add the precision there weighted by that step. A random ranking has precision π at every recall, so its AP is π, 0.038 here: AP is read against the positive rate, never against 0.5, and quoted with the positive rate beside it.

Worked example — average precision of a seven-drive ranking

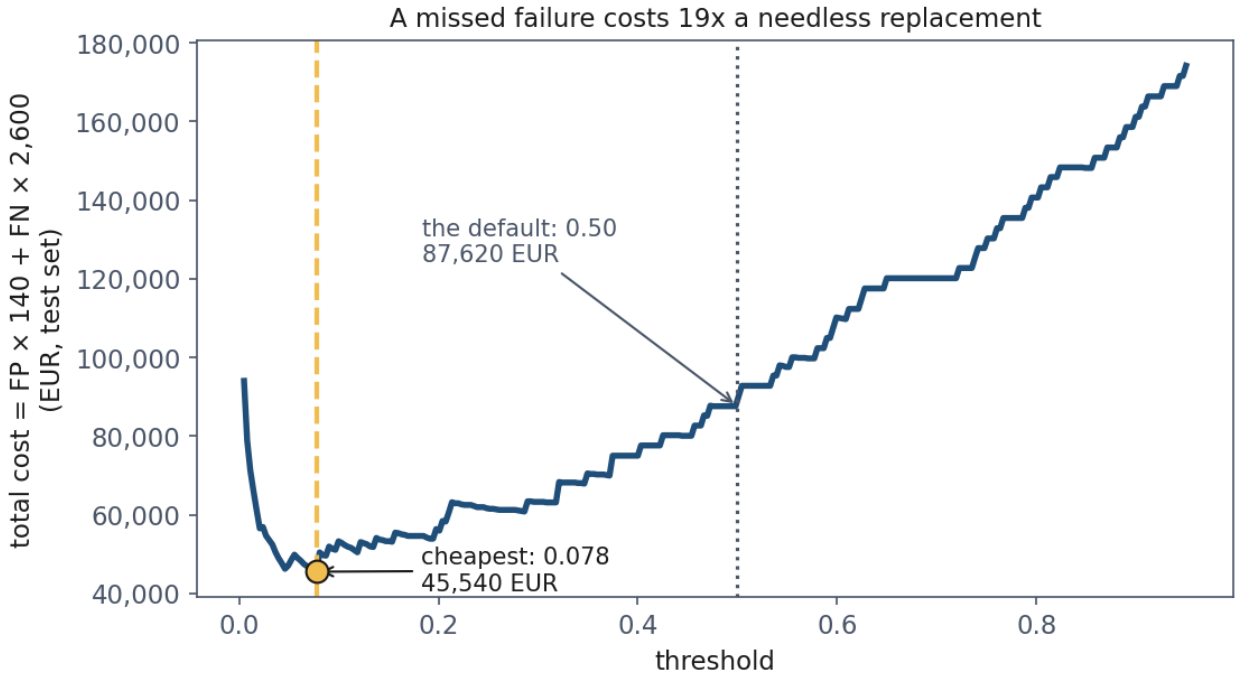
Seven drives sorted by score, riskiest first: failed, healthy, failed, healthy, healthy, failed, healthy. Recall rises only when the cut passes a failed drive, so only those cuts add a step:

cut after drive	precision there = failed ÷ flagged	recall there = failed ÷ 3	recall step	step × precision
1	1 ÷ 1 = 1.000	1 ÷ 3	1/3	1/3 × 1.000 = 0.333
3	2 ÷ 3 = 0.667	2 ÷ 3	1/3	1/3 × 0.667 = 0.222
6	3 ÷ 6 = 0.500	3 ÷ 3	1/3	1/3 × 0.500 = 0.167

AP = $\frac{1}{3} \times 1.000 + \frac{1}{3} \times 0.667 + \frac{1}{3} \times 0.500 = 0.722$. The steps are all equal, so this is also the plain average of the three precisions: $(1.000 + 0.667 + 0.500) \div 3 = 0.722$, the precision at each failing drive's place in the ranking, averaged. A random ranking would have precision equal to the positive rate at every cut, $3 \div 7 = 0.429$, and so an AP of 0.429: that, not 0.5, is the line this ranking has to beat.

Part VIII — The threshold from money

Slide 53 — Choosing the threshold from money



Total cost on the test set as the threshold moves, FP × 140 + FN × 2,600. The minimum is far below 0.5, and the curve is nearly flat around it.

The figure turns the costs of slide 36 into a curve: the total bill on the test set, FP × 140 + FN × 2,600, at every threshold. At 0.5 it is €87,620; the minimum, at 0.078, is €45,540, about half, with the same model and only the threshold moved. The curve is nearly flat around its minimum, so the exact value matters little, which is fortunate, because cost estimates are never exact.

Question — the threshold that minimises the real cost, and the costs come from the application?

> *Moving the threshold moves FP and FN; the minimum cost is near 0.08. The costs do not come from the model, they come from the specific application.*

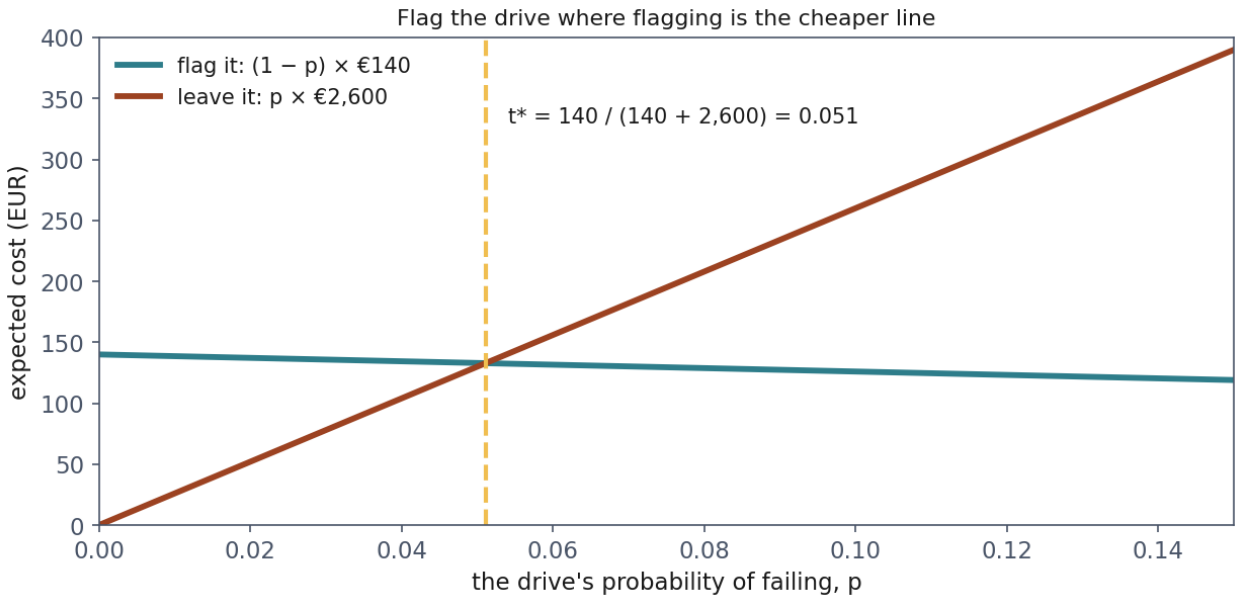
Right. At 0.5 the bill is 13 × 140 + 33 × 2,600 = 1,820 + 85,800 = €87,620. At the cheapest threshold, 0.078 (printed with three decimals; at exactly 0.08 the cost is already higher), it is 121 × 140 + 11 × 2,600 = 16,940 + 28,600 = €45,540: about half, 45,540 / 87,620 = 0.52, with the same model. Going down from 0.5, each miss at €2,600 becomes a false alarm at €140, a good trade until false alarms pile up near 0; at high thresholds the cost climbs towards doing nothing, 76 × 2,600 = €197,600. Each little step is one drive crossing the threshold.

Two conditions. The 0.078 was found by looking at the test set (slide 56 deals with that). And the flat bottom is good news: the exact value matters little, which is fortunate, because cost estimates are never exact.

Question — must the threshold always come from costs?

No. Costs are one principled criterion, and the one used here. Others can be just as defensible: a capacity ("the technicians can follow up so many alarms a week", so flag the top of the ranking), or a required recall. What is not defensible is no criterion at all, the 0.5 nobody chose. The homework has a capacity constraint for exactly this reason.

Slide 54 — Where the two costs cross



One drive's expected cost against its probability p: flagging (teal), $(1-p) \times 140$; leaving (rust), $p \times 2600$. Flag every drive to the right of the crossing.

The picture before the formula. Flagging costs €140 only if the drive was healthy, probability $1-p$: a nearly flat line. Leaving costs €2,600 only if it fails, probability p: a steep line. Left of the crossing, leaving is cheaper; right of it, flagging is. If a failure also cost €140, the two lines would have opposite slopes of the same size and cross at 0.5: the default threshold is right only when the costs are equal.

Slide 55 — Where they cross, in one line

The crossing of slide 54, in algebra. Flag a drive when leaving it costs more in expectation than flagging it: $p\,C_{FN} > (1-p)\,C_{FP}$. Solved for p, the drive is flagged when $p > C_{FP}/(C_{FP}+C_{FN})$, which with €140 and €2,600 is $140 \div 2{,}740 = 0.051$. That threshold, t^*, depends only on the two costs, not on the model or the data, and it holds on one condition: that p is calibrated.

Question — how is the formula reached? The probability is in the notes but not in the formula

> *It is not clear how the formula is reached; the notes talk about the probability, which then does not appear in the formula, maybe because it simplifies. And why is the theoretical 0.051 different from the 0.078 found on the test set?*

The probability does not simplify away: it is the unknown being solved for, and t^* is a value of p. Flag a drive when leaving it costs more in expectation: p × 2,600 > (1 - p) × 140, so 140 < p × 2,740, so p > 140 / 2,740 = 0.051. t^* is the p at which the two expected costs are equal, the crossing of slide 54, and the rule reads "flag every drive with p above t^*". Read the formula before the number: $t^* = C_{FP}/(C_{FP}+C_{FN})$ depends only on the ratio of the two costs, not on the model, the dataset or the class balance. Its condition is calibration: p must be an honest probability.

The gap between 0.051 and 0.078 is counted in notebook 3. 45 test drives are scored between the two, and 1 of them failed. Flagging them would cost $44 \times 140 =$ €6,160; leaving them cost that one failure, €2,600, so on this test set leaving them was cheaper. But their probabilities add up to 2.76 expected failures: for a calibrated model, leaving them costs about $2.76 \times 2{,}600 =$ €7,176 against $(45-2.76) \times 140 =$ €5,914 for flagging, and the rule is right. The gap is 45 drives with one failure where the model expected almost three: sampling noise on 76 failures, plus calibration that is good but not perfect. At t^* itself the cost is €49,100, 8% above the minimum, on the flat bottom of slide 53.

Question — on an imbalanced problem, does the threshold always belong below 0.5?

No, although every threshold in this lesson moves down, which makes the pattern easy to generalise. The class balance is not in the formula. With calibrated probabilities and equal costs, t^* is 0.5 however rare the failures are. The threshold falls here because a miss costs 18.6 false alarms, not because failures are rare.

Worked example — invented costs: a false alarm €50, a missed failure €200

Suppose $C_{FP} = 50$ and $C_{FN} = 200$, in euros: round numbers made up for the example, not the lesson's 140 and 2,600. The formula of slide 55 gives $t^* = C_{FP} \div (C_{FP} + C_{FN}) = 50 \div (50+200) = 0.20$. Check it on two drives, comparing the two expected costs directly:

drive's p	expected cost of flagging = $(1-p) \times C_{FP}$	expected cost of leaving = $p \times C_{FN}$	decision: the cheaper
0.15	$(1 - 0.15) \times 50 = 42.5$	$0.15 \times 200 = 30.0$	**leave**
0.3	$(1 - 0.3) \times 50 = 35.0$	$0.3 \times 200 = 60.0$	**flag**

The drive at 0.15 is below t^* and is left alone; the one at 0.3 is above and is flagged, and the direct comparison agrees both times. Make the miss cost 50 as well and $t^* = 50 \div (50 + 50) = 0.5$, the default.

Slide 56 — The caveat you must state

The slide names the flaw in what was just done. The cheapest threshold, 0.078, was found by trying thresholds on the test set, which is the step lesson 1 forbade: once the test set has helped choose something, its score is no longer an honest estimate. Done properly, the threshold is chosen on a validation set and the test set is used once, at the end (lesson 5). The price of looking, measured: $t^* = 0.051$, fixed by the costs before looking, costs €49,100 on the test set; 0.078 costs €45,540.

Question — we chose the threshold by looking at the test set: validation set, then test once?

> *We are choosing the threshold by looking at the test set, so we broke lesson 1's rule. It should be chosen on a validation set, then used on the test set to get an unbiased estimate.*

Right, with the condition on "unbiased": the test score is an unbiased estimate only if the test set played no part in any choice and is used once, at the end. How much the peeking is worth here: t^*, fixed by the costs before anyone looked, costs €49,100 on the test set; 0.078, the best of 300 thresholds tried on it, costs €45,540. The €3,560 between them (8%) is partly luck the test set handed us, the 45 drives with one failure where almost three were expected; the minimum of many tries on the same data is biased downwards. Until lesson 5 provides a validation set, treat 0.078 as a demonstration of a method, not a result anyone could publish.

Question — if 0.078 is cheaper, why not deploy it?

The slide's last bullet answers it: deploy 0.051, or a threshold chosen on validation data. "Cheaper" was measured on the same drives it was chosen on. On new drives its advantage over t^* would most likely shrink or vanish; the only way to know is to choose on one set and measure on another.

Slide 57 — What class weights do

This slide is optional in class: slide 58 carries its message. A second lever against imbalance, pulled at training time instead of after it. A **class weight** multiplies each training drive's log-loss term, so a heavily weighted class pulls harder on the fit. These class weights are not the weights w of slide 12: they say how much each example counts in the cost, while w is what the fitting chooses. `class_weight="balanced"` sets each class's weight to m ÷ (2 × class size): 0.52 for a healthy drive, 13.0 for a failed one, so a failure counts as 25 healthy drives. The name says what this does: weight times class size is the same for both classes, 0.52 × 5,770 and 13.0 × 230, both about 3,000, which is m ÷ 2, so each class carries half of the total weight. In effect, as the last bullet says, it only lowers the threshold: at 0.5 the weighted model decides like the plain one at 0.038, on 1,987 of the 2,000 test drives.

Question — "balanced" weights: the weights set from the costs of the two errors?

> *I think balanced weights means weights set from the real costs of FP and FN. Explain the class weights.*

A natural reading, since both are weights and both push towards the failures, but no: "balanced" uses the **class frequencies**. During training each drive's log-loss term is multiplied by the weight of its class, and "balanced" sets that weight to $m/(2 \times \text{class size})$: 6,000 / (2 × 5,770) = 0.52 for a healthy drive, 6,000 / (2 × 230) = 13.0 for a failed one. Each failure counts as 25 healthy drives, as if the training set held 25 copies of it, whatever a miss costs.

What it does: the intercept rises from −6.09 to −2.95, by 3.14, close to $\log 25 = 3.22$. Every odds is multiplied by about 25, so the weighted model's 0.5 lands where the plain model's $1/(1+25) = 0.038$ was. Checked on the test set: the plain model at 0.038 raises 208 false alarms and misses 8; the weighted model at 0.5 raises 205 and misses 8; the two agree on 1,987 of the 2,000 drives.

Weights from the costs exist too, as a separate choice: `class_weight={0: 140, 1: 2600}`, ratio 18.6, which at 0.5 raises 164 false alarms and misses 9, close to the plain model at t^* (165 and 10). Balanced weights land near the cost optimum here only because 25 happens to be close to 18.6.

Question — so weights in the ratio k are always the same as a threshold of 1/(1 + k)?

Approximately, and here almost exactly. The equivalence is exact for a correctly specified model and a large sample: then reweighting moves only the intercept, by $\log k$, and leaves the slopes alone. That is nearly the situation here, because the labels were generated by a logistic model in these very features. With a misspecified model, few data or a strong penalty, the weights also move the slopes and can change the ranking, and the match is only approximate.

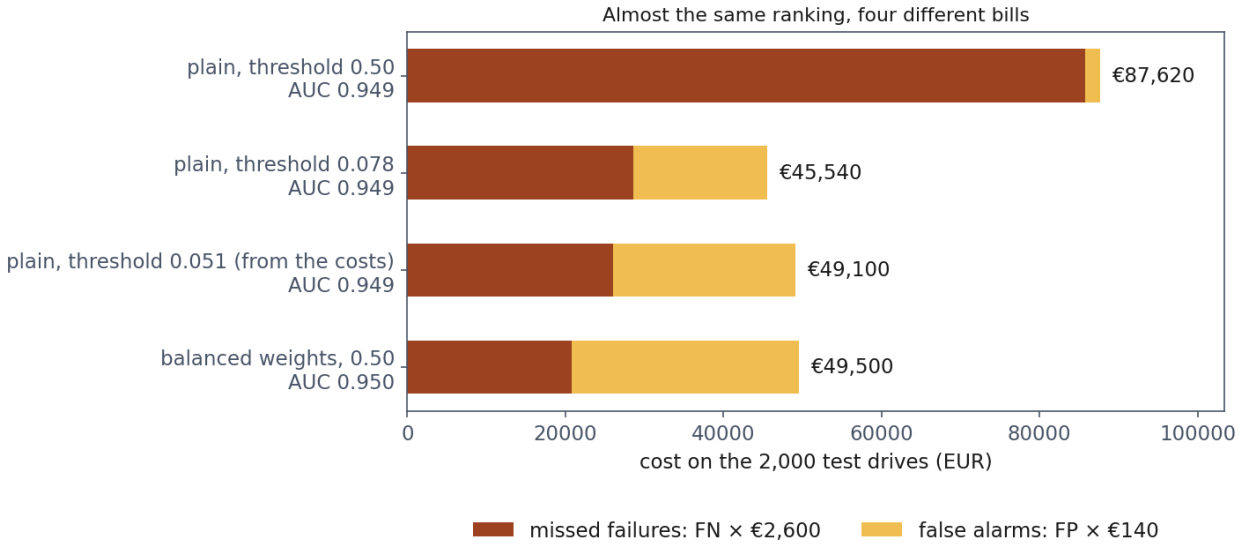
Worked example — a weight ratio of 4

Give every failure 4 times the weight of a healthy drive. For a correctly specified logistic model the main effect is on the intercept, which rises by $\log 4 = 1.386$: every odds is multiplied by 4.

Follow one drive in three steps.

1. The plain model scores it $p = 0.20$: odds $= p \div (1 - p) = 0.20 \div 0.80 = 0.25$.
2. The weighted model multiplies the odds by 4: $0.25 \times 4 = 1$.
3. Back to a probability, $p = \text{odds} \div (1 + \text{odds}) = 1 \div (1 + 1) = 0.5$. So the weighted model's threshold of 0.5 falls where the plain model's $1 \div (1+4) = 0.20$ was: the same decisions, reached by moving the line at training time instead of after it. With a misspecified model, little data or a strong penalty, the weights move the slopes too, and the match is only approximate.

Slide 58 — Class weights: the same lever, pulled earlier



Four operating points as bills: missed failures (rust, FN × 2,600) and false alarms (gold, FP × 140), with each model's AUC under its name.

The default misses 33 and pays €87,620; the threshold at 0.078 pays €45,540; the threshold fixed from the costs, 0.051, misses 10, raises 165 false alarms and pays €49,100; balanced weights miss 8, raise 205 false alarms and pay €49,500. The fair comparison for the weighted model is the 0.051 bar, not the 0.078 one, which was chosen by looking at the test set (slide 56): and against 0.051 the two cost practically the same. The AUC under each name, 0.949 against 0.950, says reweighting taught the model essentially nothing new about disk failure: it moved where the model puts its 0.5 line. That is why class weighting is not a remedy for imbalance; the remedy, where one exists, is more positive examples. It earns its place when a tool insists on `predict()` and offers no threshold to move.

Question — and resampling, or the synthetic minority oversampling technique?

The same argument: oversampling the failures, or synthesising new ones with the synthetic minority oversampling technique (SMOTE), moves the operating point too. Both must touch the training

rows only, inside each cross-validation fold once lesson 5 introduces folds. Synthetic or duplicated failures in the test set would be scored as if they were real drives, and the test score would stop describing the real fleet; lesson 2 made the same argument for fitting the imputer on the training rows only.

Slide 59 — Notebook 3, live

Notebook 3 draws the ROC curve, checks the AUC with 200,000 random pairs, runs the thinning experiment, the cost curve and the class weights, and ends with the three-class model, alongside slides 60 to 62.

Question — is the AUC bounded between 0.5 and 1?

Between 0 and 1. 0.5 is a random ranking; below 0.5 the ranking is systematically reversed, and flipping every prediction would turn an AUC of 0.2 into 0.8. In practice a model below 0.5 has a bug, usually the classes swapped.

Question — why does the notebook print the scores, the pairs and the counts, and not just the AUC?

Because every number quoted on the slides must exist in a saved notebook output: the 7,409 wrong pairs, the 1,408 healthy drives above the worst failure, the 304 false alarms in every fleet, the 45 drives between t^* and 0.078. A number that only lives in prose can drift with nothing to catch it; one printed by a cell is checked every time the notebook runs.

Part IX — More than two classes (optional)

Slide 60 — Three classes: softmax (optional)

Slides 60 to 62 are optional: exercise 4 is a two-class problem and needs none of them, and lesson 9 introduces the softmax again for the handwritten digits. With three classes, healthy, degraded and failed, one probability no longer suffices. The model computes one linear score per class, $z_k = w_k^\top x + b_k$, and the **softmax** turns the three scores into three probabilities that add up to 1, $P(\text{class } k) = e^{z_k} \div \sum_j e^{z_j}$: exponentiate each, divide by their sum. The predicted class is the one with the largest probability; there is no 0.5 threshold. With two classes the softmax is the sigmoid of $z_1 - z_0$, so this is the same model, generalised. In code, `decision_function` returns the scores and `predict_proba` the softmax.

Question — three probabilities adding to 1: where do they come from, and how is the class chosen?

> *A confusion matrix for three classes; the model produces three probabilities that add up to 1. Softmax is named in the notes but not on the slide. It comes back in lesson 9, but maybe more detail here?*

One linear score per class, $z_k = w_k^\top x + b_k$, each with its own coefficients. The softmax turns the scores into probabilities in two moves: exponentiate, so all are positive; divide by their sum,

so they add up to 1. The largest score keeps the largest share, and the predicted class is the largest probability, with no 0.5 threshold: notebook 3 counts 33 test drives assigned a class whose probability is below 0.5. A real drive shows why degraded is the hard class: a degraded drive with scores 0.615, 0.603, −1.218 gets probabilities 0.466, 0.460, 0.074 and is called healthy, by 0.006. With two classes, subtract z_0 from both scores and the softmax becomes the sigmoid of $z_1 - z_0$: the same model, seen from further away. Its cost is the categorical cross-entropy, minus the log of the probability given to the true class.

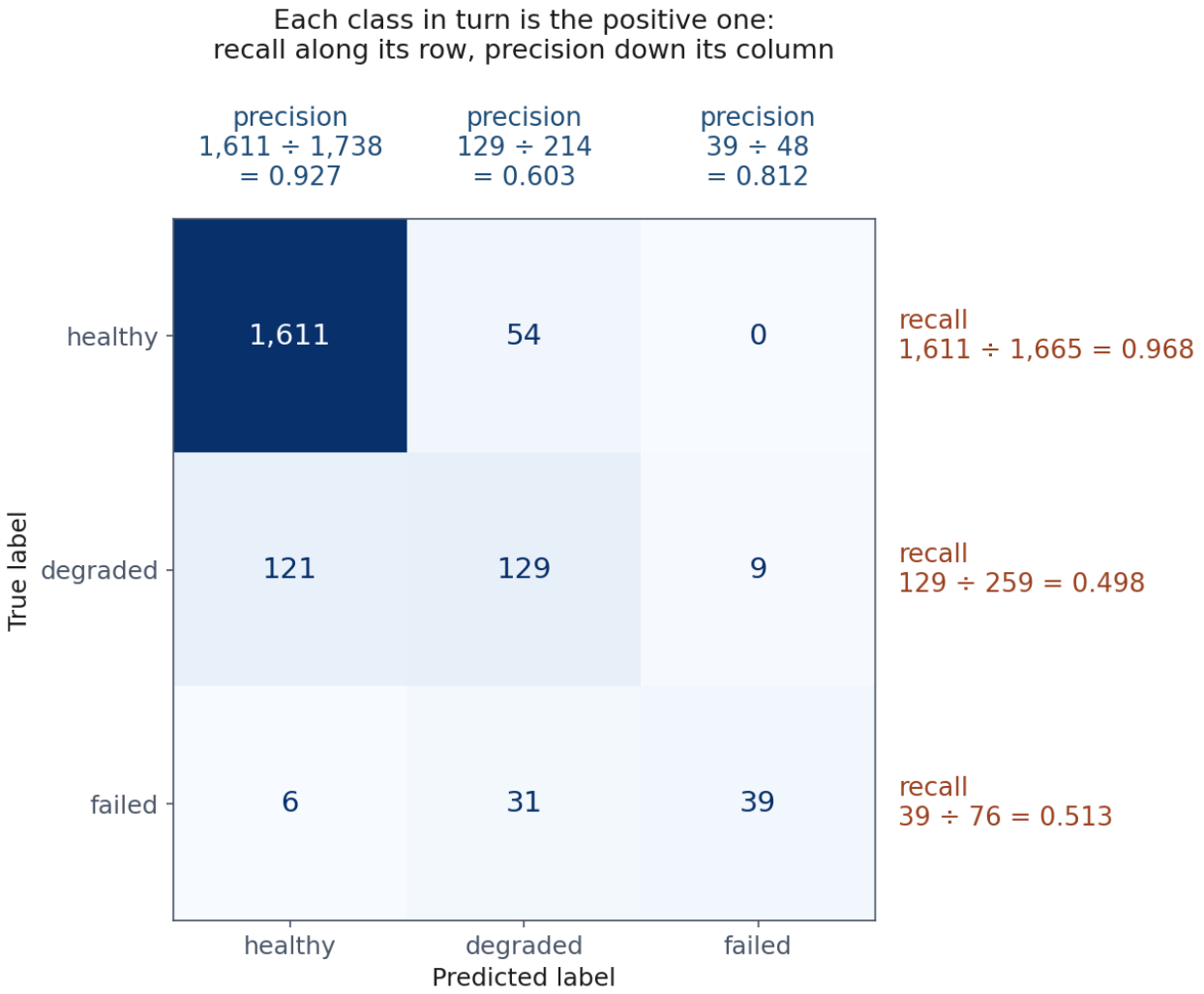
Worked example — softmax on three scores

class	score z_k	exponentiated, e^{z_k}	probability $= e^{z_k} \div$ sum
healthy	2.0	$e^{2.0} = 7.389$	$7.389 \div 11.213 = 0.659$
degraded	1.0	$e^{1.0} = 2.718$	$2.718 \div 11.213 = 0.242$
failed	0.1	$e^{0.1} = 1.105$	$1.105 \div 11.213 = 0.099$

The sum in the last column is $7.389 + 2.718 + 1.105 = 11.213$.

The three probabilities add up to 1, and the largest score keeps the largest share: the drive is called healthy with 0.659. With two classes and scores 0 and 1.5, softmax gives $e^{1.5} \div (1 + e^{1.5}) = 0.818$, which is $\sigma(1.5)$: the sigmoid is softmax with one score fixed at 0.

Slide 61 — More than two classes (optional)



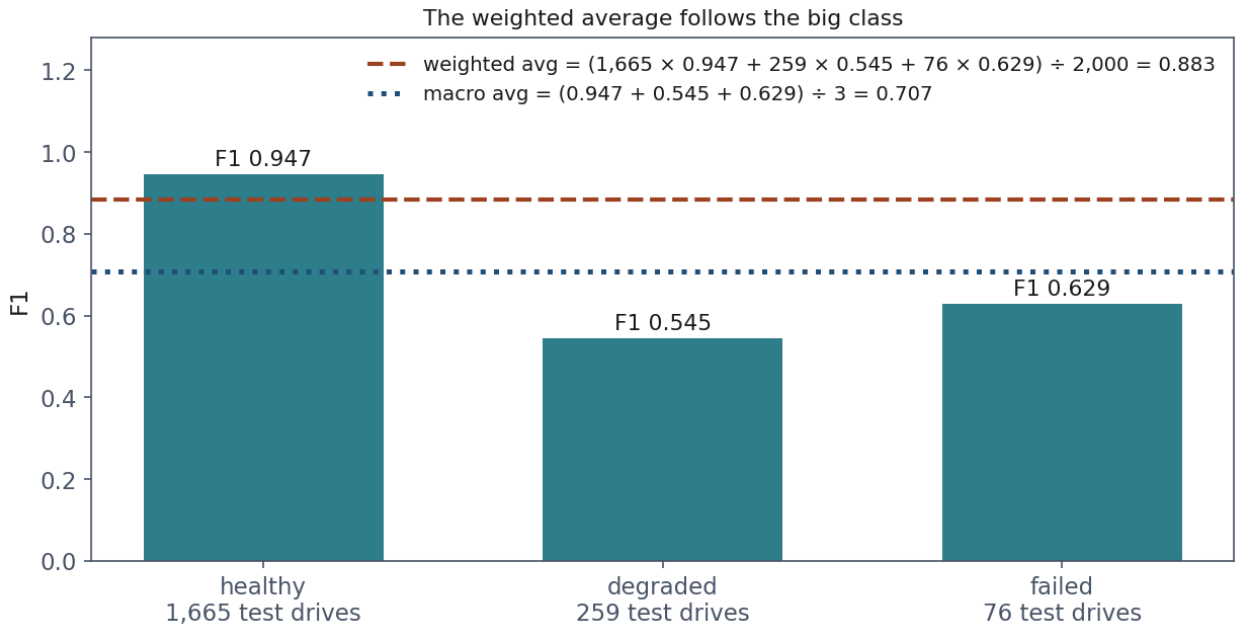
Three classes. Read the off-diagonal cells: degraded drives are confused with both neighbours. Each class's recall, on the right, is its diagonal cell divided by its row; its precision, above, is the diagonal cell divided by its column.

The confusion matrix grows to three by three: truth in rows, prediction in columns, the right calls on the diagonal, every error off it. Degraded sits between two neighbours that both look like it from a distance: 121 degraded drives were called healthy, and 31 failed drives were called degraded.

Question — how are precision and recall defined with three classes?

One class at a time: each class in turn is the positive one and the other two together are the negative one, and the binary formulas apply. That gives one precision, one recall and one F1 per class, which the next slide averages.

Slide 62 — Macro against weighted, one last time (optional)



F1 per class, with the macro average (dotted) and the weighted average (dashed).

Per-class F1 on the three-class problem, with the two averages of slide 40 drawn as lines. Healthy scores 0.947, degraded 0.545, failed 0.629. The macro average gives each class one vote; the weighted average follows the healthy class, which holds most of the drives. The gap between the two lines is the finding of the slide: the small classes are handled much worse than the big one.

Question — the weights are the number of drives per class, and the choice depends on the application?

> *F1 per class, then the macro and the weighted average, whose weights are the numbers of drives in each class. Which to prefer depends on the application, not on statistics.*

Right. The weights are the support, the number of test drives whose **true** class is that one: 1,665 healthy, 259 degraded, 76 failed. Macro: $(0.947 + 0.545 + 0.629)/3 = 0.707$. Weighted: $(1{,}665 \times 0.947 + 259 \times 0.545 + 76 \times 0.629)\ /\ 2{,}000 = 0.883$. The gap, $0.883 - 0.707 = 0.18$, is itself the finding: the small classes are handled much worse than the big one. A question that may come: the weighted average of recall always equals the accuracy, 0.889 here, because support times recall is the number of drives of that class classified correctly.

Slide 63 — What to take away

Five lines to keep. Logistic regression is the sigmoid of a linear score, fitted by log loss. Accuracy means nothing without its baseline: 96.20% for a model that catches nothing, 1,924 ÷ 2,000. Precision is about the alarms, recall about the failures. The threshold comes from the costs, 140 ÷ 2,740 = 0.051, if the probabilities are calibrated. And the AUC measures the ranking only: it is blind to calibration and hides imbalance.

Question — accuracy alone says little; precision for alarms, recall for failures; the threshold from costs; AUC for ranking?

Right, all four, with the conditions the bullets carry: accuracy says little **alone**, but against its baseline it says something (97.70% against 96.20%); the formula $t^* = C_{FP}/(C_{FP} + C_{FN})$ holds for calibrated probabilities; and the AUC is blind to calibration as well as to imbalance. The first bullet is the first half of the lesson in one line: log loss is what maximum likelihood gives, and unlike squared error through the sigmoid it is convex, so any minimum gradient descent finds is the global one.

The reflex to keep: before believing any accuracy, ask for the base rate.

Slide 64 — Homework

Exercise 4 gives a new imbalanced dataset, patients who may be readmitted to hospital, and asks for a classifier, a threshold chosen from a stated cost ratio, and a paragraph defending the choice. It is discussed at the start of the lesson on Friday 13 November. What matters at the exam is the paragraph, not the classifier, which is a few lines anyone can write. Two traps are planted on purpose: the dataset has a decoy column, as this one did, and the exercise asks for the AUC and the average precision together; reporting the AUC alone would describe a model better than it is. And the clinic has a limited capacity that no cost formula knows about: the defensible threshold is not necessarily the cheapest one.